mathematics

MDPI

Article

A Population-Based Iterative Greedy Algorithm for Multi-Robot Rescue Path Planning with Task Utility

Mingming Li and Peng Duan *

School of Computer Science, Liaocheng University, Liaocheng 252059, China; 2310170106@stu.lcu.edu.cn
* Correspondence: duanpeng@lcu.edu.cn

Abstract

Multi-robot rescue path planning (MRRPP) is critical for ensuring the rapid and effective completion of post-disaster rescue tasks. Most studies focus on minimizing the length of rescue paths, the number of robots, and rescue time, neglecting the task utility, which reflects the effect of timely emergency supplies delivery, which is also important for post-disaster rescue. In this study, we integrated multiple optimization indicators into the rescue cost and modeled the problem as a variant of the vehicle routing problem (VRP) with timeliness and battery constraints. A population-based iterative greedy algorithm with Q-learning (QPIG) is proposed to solve it. First, two problem-specific heuristic schemes are designed to generate a high-quality and diverse population. Second, a competition-oriented destruction-reconstruction mechanism is applied to improve the global search ability of the algorithm. In addition, a Q-learning-based local search strategy is developed to enhance the algorithm's exploitation ability. Moreover, a historical information-based constructive strategy is investigated to accelerate the convergence speed of the algorithm. Finally, the proposed QPIG is validated by comparing it with five efficient algorithms on 56 instances. Experiment results show that the proposed QPIG significantly outperforms compared algorithms in terms of rescue cost and convergence speed.

Keywords: multi-robot rescue path planning; population-based iterative greedy algorithm; competition-oriented destruction-reconstruction; Q-learning-based local search; historical information

MSC: 93B28

1. Introduction

With the increasing frequency of natural disasters, the security and stability of human society are facing severe challenges [1]. In recent years, post-disaster relief has become an important research field [2]. In some severely affected regions, high-risk factors such as structural collapse and leakage of hazardous substances make it difficult for rescue workers to enter the disaster area safely [3]. The development of robot technology provides a new solution for post-disaster rescue. Rescue robots are capable of replacing or assisting humans in search, positioning, material transportation, and other tasks in dangerous environments [4]. However, the post-disaster rescue tasks are usually numerous and widely distributed, and a single robot is difficult to meet the actual needs. Multi-robots are able to work together to complete complex rescue tasks efficiently [5]. Therefore, how to effectively design rescue sequences for multiple robots in post-disaster rescue is an urgent problem to be solved.

Check for updates

Academic Editor: Janez Žerovnik

Received: 27 November 2025
Revised: 25 December 2025
Accepted: 29 December 2025
Published: 31 December 2025

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

Currently, the multi-robot path planning (MRPP) research mainly focuses on the shortest path or the minimum energy consumption and has made significant progress [6,7]. Many researchers model these problems as variants of the VRP to optimize the path of the robot. For example, Cui et al. [8] introduced a mathematical model for the rescue vehicle routing problem with time windows and limited survival time, which aims to minimize the total distance traveled by helicopters while considering time windows, survivor life strength, and the capacities of transport and medical helicopters. Lin et al. [9] designed a grouping method in disaster environments to minimize task failures and rescue time based on task locations and time constraints. Geng et al. [10] proposed a modified PSO algorithm to maximize the number of successfully rescued survivors by assigning tasks to robots while considering uncertain survival times, ensuring that each robot operates within time constraints and optimizes task efficiency. Huang et al. [11] proposed a model to minimize both path cost and time cost to complete the tasks in the shortest time, while ensuring that each robot has enough supplies to perform its assigned task. Wang et al. [12] developed an improved IG algorithm to maximize the number of rescued survivors by considering life strength constraints and the time robots take to reach each survivor. Although existing research in rescue problems often considers optimization objectives such as path length, time, and the number of robots, most studies overlook the critical factor of task utility. Task utility refers to the rescue effect of the timely delivery of emergency supplies, and it decays exponentially over time. If emergency supplies can be delivered promptly and assistance can be provided, the task utility will be extremely high. If delivery is delayed, even if the supplies are intact, their utility value will decrease. Therefore, path planning should not only optimize the shortest distance and time but also maximize task utility to enhance rescue effectiveness. Additionally, considering that rescue robots have limited battery, path planning must also account for battery usage to ensure that robots can effectively complete their tasks. Therefore, based on the traditional VRP model, a new problem model is established considering the timeliness of the task and the battery constraint of the robot. The optimization objective of the model is composed of path length, task utility, number of robots, and rescue time. It aims to achieve efficient path planning by minimizing the objective function.

The MRPP problem is essentially an NP-hard problem, as demonstrated by reductions from well-established NP-hard problems such as the vehicle routing problem (VRP) and the path planning problem [13–16]. This makes it challenging to achieve the optimal solution using exact algorithms within a specified time. Using heuristics and metaheuristics is one of the most efficient algorithms for finding the optimal or approximate optimal solution to NP-hard problems [17]. As a straightforward and efficient metaheuristic algorithm, the iterative greedy (IG) algorithm offers the benefits of a simple structure, minimal parameter requirements, and embeddability. The IG algorithm has been extensively used to solve combinatorial optimization problems [18–21]. The MRRPP problem and combinatorial optimization problems have similarities in solution structure. Both problem types require identifying the optimal sequence among a large number of possible configurations. Therefore, this study explores the applicability of the IG algorithm in the MRRPP problem. However, given the features of the MRRPP problem, the traditional IG algorithm is limited by its single-solution greedy strategy and is susceptible to becoming trapped in local optima when handling large-scale problems. Therefore, this work proposes an improved population-based iterative greedy algorithm for solving the MRRPP problem. Specifically, a history-based exploration strategy, a collective intelligence-driven strategy, and an elite-oriented optimization strategy are designed to enhance solution diversity. Additionally, six neighborhood operators based on critical and non-critical paths are developed for further exploration, and Q-learning is investigated to select these operators, thereby enhancing the

algorithm's effectiveness in exploring the search space. The primary contributions of this study are as follows:

(1) A novel mathematical model for the MRRPP problem is proposed, emphasizing the integration of time-decaying task utility to optimize rescue performance.
(2) A competition-oriented destruction-reconstruction mechanism is designed to improve the QPIG's global search ability.
(3) A Q-learning-based local search strategy is developed to increase the exploitation ability of the algorithm.
(4) A historical information-based constructive strategy is investigated to accelerate the algorithm's convergence speed.

The remainder of this study is structured as follows. Section 2 reviews the related literature. Section 3 explains the formulation and description of the problem. Section 4 provides a detailed explanation of the proposed QPIG algorithm. Section 5 describes parameter calibration, effectiveness analysis, and comparative algorithms for the experiment. Finally, Section 6 summarizes this study and presents possible directions for further research.

2. Literature Review

In recent years, several algorithms have been utilized by many researchers to solve rescue path planning problems. For example, Xu et al. [22] proposed an obstacle constraint and task of equal division-based k-means clustering algorithm, along with a glow-worm swarm optimization algorithm with chaotic initialization (GSOCI), which significantly improves the accuracy and convergence speed in addressing the emergency rescue planning issue. Michael et al. [23] applied a variant of the ACO algorithm to determine the optimal search path under time and resource constraints, aiming to maximize the probability of locating survivors. Additionally, Wen et al. [24] developed an unmanned aerial vehicle (UAV) route planning method that incorporated a probabilistic model to estimate potential target locations and two search algorithms to maximize detection probability in solving maritime search and rescue challenges. Meanwhile, Alrayes et al. [25] used an enhanced drone route planning scheme that integrates quasi-oppositional-based learning with the traditional search-and-rescue optimization algorithm to optimize secure communication flight paths. Liu et al. [26] implemented a multi-objective path planning framework, using hierarchical target filtering and sorting strategies to improve efficiency in complex disaster scenarios. Although the aforementioned algorithms have achieved notable results in rescue path planning, there remains room for improvement in balancing solution quality and computational efficiency, especially in large-scale environments.

Given its efficiency and ability to generate near-optimal solutions, the IG algorithm has been extensively adopted to address combinatorial optimization problems since its introduction. In the past several years, many variants of IG have been developed, among which the most notable is the population-based iterative greedy (PIG) algorithm. For instance, Bouamama et al. [27] were the first to establish the PIG algorithm for the minimum weight vertex cover problem. Miao et al. [28] used a PIG algorithm enhanced with Q-learning for solving the scheduling issue of multiple weeding robots, focusing on minimizing completion time while lowering total operational costs. In addition, Zou et al. [29] implemented a mixed-integer linear optimization model and PIG algorithm to tackle the scheduling of automated guided vehicles with unloading safety detection in a matrix workshop. Wang et al. [30] designed the PIG algorithm to address the cascaded flowshop joint scheduling problem, where the search alternates between two sequential phases to effectively reduce total tardiness. Moreover, Zhao et al. [31] presented the PIG algorithm, incorporating heuristic initialization, local search, and selection mechanism, and achieving superior performance on extensive instances. Wang et al. [32] developed two

rapid evaluation approaches and a modified idle time insertion method within the PIG algorithm, resulting in notable improvements in both efficiency and solution robustness. Therefore, considering its verified efficiency in solving various combinatorial optimization problems, the PIG algorithm is employed as the core framework to balance search intensification and diversification.

Reinforcement learning (RL) has attracted growing interest recently, prompting researchers to investigate its potential for tackling optimization problems. To improve solution quality in multi-objective optimization, Li et al. [33] designed a Q-learning-based dimension detection search method for identifying promising non-dominated solutions in resource-constrained flexible flowshop scheduling with robotic transportation. Zhao et al. [34] introduced a Q-learning-based approach for determining the weighting coefficients, aiming to achieve smaller objective values in the distributed no-idle permutation flowshop scheduling problem. Ren et al. [35] used an ensemble artificial bee colony algorithm enhanced with Q-learning to solve the bi-objective disassembly line scheduling problem, optimizing both disassembly time and smoothing index while considering task interference. In addition, Luo et al. [36] developed a Q-learning memetic algorithm (QLMA) that considers product priorities and factory heterogeneity to minimize energy consumption and total tardiness. To address the multi-UAV cooperative plant protection task allocation problem, Chen et al. [37] investigated a learning-based memetic algorithm (L-MA) that utilizes Q-learning to assist operators in determining the optimal number of UAVs for each field. Moreover, Fang et al. [38] suggested an improved ant colony optimization algorithm (IAC-IQL), integrated with enhanced Q-learning, for global path planning of search and rescue robots based on Bessel curves, achieving higher efficiency and smoother paths. Zhan et al. [39] developed a reinforcement learning-based genetic algorithm (GA-RL), which leverages Q-learning to guide offspring selection and population renewal in UAV maritime search and rescue involving multiple rescue centers. Inspired by the successful integration of Q-learning for scheduling and path planning problems, this study employs Q-learning to intelligently steer the choice of operators, allowing the algorithm to learn from experience and improve solution quality.

According to the algorithm described above, we chose nine representative algorithms and detailed their advantages and limitations in Table 1. Through comparison, we found that swarm intelligence algorithms such as GSOCI, IACO, DABC, QLMA, IAC-IQL, and GA-RL have more parameters, complex population structures, and higher computational complexity. In the MRRPP problem, we usually only need an optimal path solution. Compared to swarm intelligence algorithms, the improved IG algorithm can explore the solution more thoroughly and effectively optimize its quality. Based on the advantages of the IG algorithm and the current research, this work introduces further improvements on the basis of the IG algorithm.

The traditional IG algorithm performs excellently in solving small-scale problems. As the number of rescue points increases, the advantages of its iterative improvement strategy gradually reduce, and the algorithm may overly focus on certain areas, leading to getting stuck in local optima. This work proposes a competition-oriented destruction-reconstruction mechanism to improve global search capability. Additionally, the traditional IG algorithm only uses insertion operators, resulting in incomplete exploration of the solution space. Therefore, this work designed six neighborhood operators based on the critical path and non-critical path, expanding the search range and enabling more comprehensive exploration of the solution space.

Table 1. Analysis of related algorithms.

Algorithms	Advantages	Limitations
GSOCI [22]	The initial distribution is enhanced by using chaotic initialization and Chebyshev mapping to improve population diversity.	The initial clustering effect directly determines the quality of path planning.
IACO [23]	Propose an effective restart strategy to achieve diverse search.	The parameter configuration is complex, and the optimal parameter combination varies significantly across different scenarios.
IIG [12]	A greedy-based insertion strategy is employed to improve the algorithm's exploration capability.	Rescue operations are restricted to a single robot.
Q_DPIG [28]	Select destruction operators adaptively to evaluate the search condition.	The algorithm executes several destruction operations, resulting in high computational complexity.
PBIG [29]	Two-stage destruction strategy to avoid local optima, dual selection reconstruction strategy to obtain better neighborhood solutions.	It is prone to getting trapped in local optima for complex problems.
ABC_QL [35]	Two Q-learning-based strategies are employed to select high-quality operators.	The algorithm converges slowly when the solution space complexity is high.
QLMA [36]	An efficient neighborhood structure guides the population towards better convergence.	The global search capability is weak, making it prone to getting stuck in local optima.
IAC-IQL [38]	A heuristic learning evaluation model dynamically adjusts the learning factors to guide the search path.	The time complexity of the algorithm is not considered.
GA-RL [39]	The individual retention mechanism is used to update the population based on the performance of individuals in the elite depot.	The local search is insufficiently performed, and the solution space is not fully explored.

3. Problem Description and Formulation

3.1. Problem Description

In post-disaster rescue scenarios, there are multiple rescue points, assuming each rescue point corresponds to a task. Robots are mainly responsible for carrying out material distribution tasks from rescue centers to various rescue points, usually involving the delivery of emergency supplies. Tasks are usually time sensitive, and their utility gradually declines over time. The task utility refers to the effect or value of the timely delivery of emergency supplies. In post-disaster rescue scenarios, for example, the task is to deliver medication to patients. The "value" of the medication does not refer to its monetary worth as an emergency supply, but rather to the therapeutic effect it can provide. If the medication can be delivered to the patient promptly and provide treatment, its task utility will be very high, as it can significantly improve the patient's health condition and increase the likelihood of a successful recovery. If the delivery is delayed, even if the medicine is not physically damaged, its task utility value will decrease because it does not provide timely assistance, impacting the rescue effectiveness. This study formulated the rescue problem as a variation of the VRP with timeliness and battery constraints.

The problem is formulated as a directed connected graph $G = (N, A)$, where $N = \{0\} \cup T$ is the set of vertices, 0 denotes the rescue center, T is defined as the set of rescue points, and $A = \{(i, j) \mid i, j \in T, i = j\}$ is τ, the arc set composed of distances between rescue points. All rescue tasks are performed by k robots, and the robot set is $R = \{1, 2, \ldots, k\}$. Each rescue point must be accessed by at least one robot to ensure smooth completion of all tasks. Additionally, without compromising generality, the following assumptions have been made:

(1) All robots are homogeneous.
(2) There is only one rescue center, and robots are required to leave the rescue center and return after completing their tasks.
(3) All robots are operating normally without any downtime or malfunctions.

(4) The maximum load of all robots is the same.
(5) Robots move at a known constant speed.

3.2. Problem Formulation

In this section, a mathematical model is established as follows:
Decision variables:

x_{ijk} If the robot R_k goes from the rescue point T_i to the rescue point T_j, then $x_{ijk} = 1$; otherwise, $x_{ijk} = 0$.

y_{ik} If the robot R_k goes to the rescue point T_i to perform a task, then $y_{ik} = 1$; otherwise, $y_{ik} = 0$.

Objective:

$$\min f = w_l \sum_{k=1}^{R} \sum_{i=0}^{N} \sum_{j=0, j\neq i}^{N} x_{ijk} d_{ij} + w_n R + w_t(\tau_n^e - \tau_1^s) - w_r \sum_{k=1}^{R} \sum_{i=1}^{N} y_{ik} U_i e^{-\mu_i(t_{ik} - a_i)} \quad (1)$$

Constraints:

$$\sum_{j \in N} x_{0jk} = 1 \quad \forall k \in R \quad (2)$$

$$\sum_{i \in N} x_{i0k} = 1 \quad \forall k \in R \quad (3)$$

$$\sum_{i \in N} y_{ik} \leq 1 \quad \forall k \in R \quad (4)$$

$$a_i \leq t_{ik} \leq b_i \quad \forall i \in N, \ \forall k \in R \quad (5)$$

$$\sum_{i=1}^{N} y_{ik} dr_i \leq C_k^{max} \quad (6)$$

$$E_k = d_{i,k} \times E_a \quad (7)$$

$$E_k + \xi \leq Q_k \quad (8)$$

The objective function (1) consists of four parts: path length, number of robots, rescue time, and task utility. Constraints (2) and (3) indicate that each path taken by the robot starts and ends at the rescue center. Constraint (4) represents that the robot can only access each rescue point once. Constraint (5) ensures that the service time of each rescue point is within its time window. Constraint (6) guarantees that the robot's material transport does not surpass its maximum carrying capacity. Constraint (7) represents the battery consumption of the robot during task execution, which depends on the travel distance and the battery usage per unit distance. Constraint (8) indicates a battery constraint, ensuring that the combined battery usage of the robot and its backup does not exceed the robot's total battery capacity.

4. The Proposed QPIG Algorithm

The proposed QPIG is structured around four components: the population initialization, destruction and reconstruction phase, local search phase, and acceptance criteria phase. By introducing population-based optimization, multiple solution individuals are processed in parallel. The population-based IG framework not only preserves the advantage of IG in exploring a single solution in depth but also maintains high diversity in the solution space, thereby enhancing global search capability.

In the initialization phase, two problem-specific heuristic schemes (i.e., a utility-based initialization strategy and a rescue cost-based initialization strategy) are designed to establish an initial population composed of high-quality individuals. In the destruction and reconstruction phase, a competition-oriented destruction-reconstruction mechanism gener-

ates candidate solutions and selects the optimal individual from the candidate solutions to conduct local search. This mechanism includes a history-based exploration strategy, a collective intelligence-driven strategy, and an elite-oriented optimization strategy to improve the search process. By integrating into the competition-oriented destruction-reconstruction mechanism, individuals in the population undergo a process of selection, continuously filtering out the optimal solutions, which further improves search efficiency. The competition mechanism prompts the algorithm to select the best individuals, thereby accelerating the global search. In the local search phase, a Q-learning-based local search strategy is investigated to select six neighborhood operators to improve the quality of solutions. To overcome the limitations of traditional fixed selection strategies in complex problems, Q-learning adapts by selecting operators and dynamically adjusting the search strategy, allowing the algorithm to make flexible decisions based on the current solution's state and historical experience. This effectively balances global and local optimization, improving search efficiency and enhancing the quality of the solution. The proposed QPIG framework is presented in Algorithm 1. Lines 10 to 19 of Algorithm 1 describe the acceptance criteria phase, while lines 20 to 24 provide a historical information-based constructive strategy, which clears the stored historical solution set periodically after a predefined number of executions. Specifically, the historical information-based constructive strategy includes the historical path allocation operator and path refinement operator to accelerate the convergence speed of the algorithm. The algorithm continues executing the above loop until the stopping criterion is met and then outputs the optimal path. Figure 1 provides a flowchart of the QPIG.

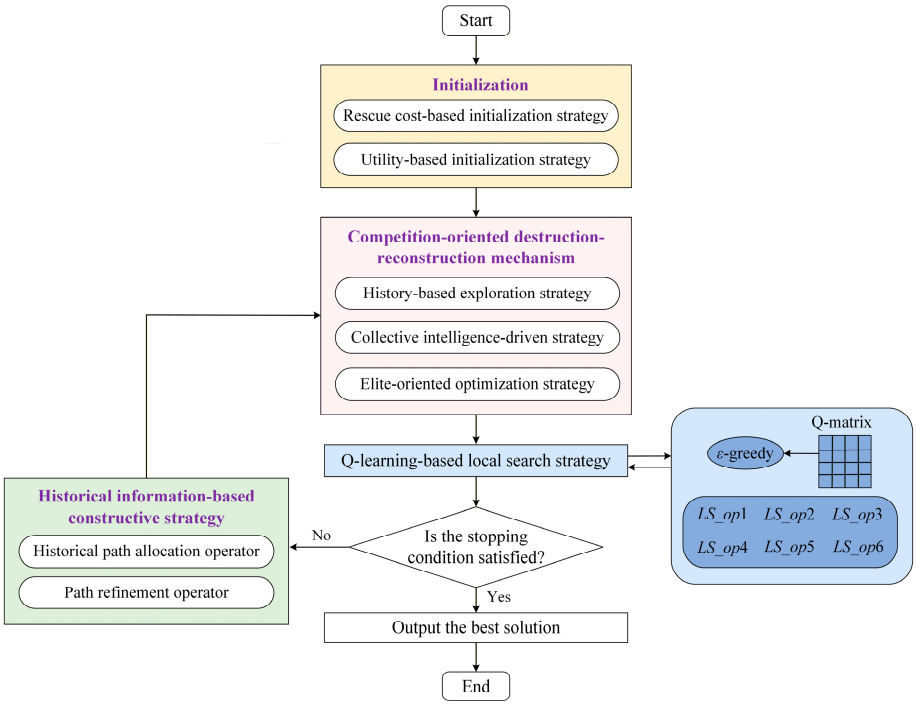


Figure 1. The flow chart of the proposed QPIG.

Algorithm 1: Framework of the proposed QPIG

Input: population size: ps, destruction-reconstruction size: l_d, simulated annealing acceptance probability: p, predefined constant: c
Output: the best solution π^{best}

```
(π, POP) = Initialization;
i = 0, RSeq = ϕ, Histsol = ϕ;
π^best = π;
while the stopping criterion is not satisfied do
    POP′ = Competition-oriented destruction-reconstruction mechanism(POP, π, l_d);
    POP″ = Q-learning-based local search strategy(POP′);
    idx_best = arg(min_{j=1}^{n}(f(POP″_j)));
    π^1 = POP″_{idx_best};
    RSeq = Rseq ∪ π^1;
    if f(π^1) < f(π) then
        π = π^1;
        if f(π^1) < f(π^best) then
            π^best = π^1;
        end
    else
        if p < rand(0,1) then
            π = π^1;
        end
    end
    if mod(i, c) == 0 then
        π_history = Historical information-based constructive strategy(RSeq);
        Add π_history to Histsol;
        RSeq = ϕ;
    end
    i = i + 1;
end
return π^best;
```

4.1. Solution Representation

Figure 2 shows an example of the MRRPP problem, where 0 represents the rescue center and {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} denote the rescue points. R_1, R_2, and R_3 represent the robots. Taking the path of robot R_1 as an example, it starts from the rescue center {0} and visits the rescue points {5, 3, 4, 1} in sequence. The t_5(6,9) next to rescue point 5 indicates that the earliest service start time of rescue point 5 is 6 and the latest service start time is 9. The demand for rescue point 5 is $dr_5 = 10$, which means 10 units of rescue supplies are required. Each time a robot visits a rescue point, its load is reduced. If the remaining load is insufficient to meet the demand at the rescue point, the robot will return to the rescue center. The initial utility value at rescue point 5 is $U = 79$, indicating that the robot's timely delivery of rescue supplies is expected to contribute a value of 79 to this rescue point. As shown in the upper right corner of Figure 2, each robot has an initial load capacity of 150 and a total battery capacity of 110. In this study, a two-dimensional array is employed to represent each solution. The first dimension of the array corresponds to the robot number, and the second dimension represents the rescue points allocated to that robot. Figure 3 illustrates an example of the encoding method.

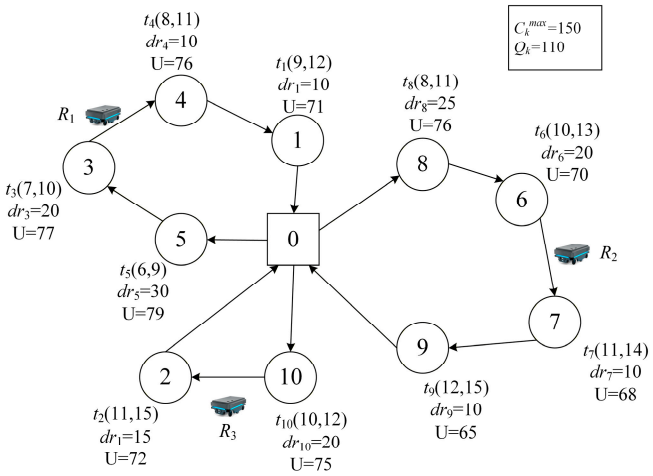


Figure 2. An example of the MRRPP problem.

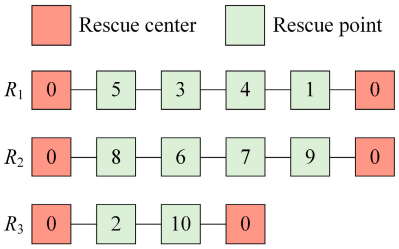


Figure 3. An example of the solution representation.

4.2. Population Initialization

An effective initial solution significantly speeds up the convergence process and improves the solution's quality. In this study, two problem-specific heuristic schemes are designed. The first heuristic scheme is the utility-based initialization strategy, which sorts tasks according to their utility values. Firstly, tasks with low utility values are assigned to each robot as the starting point. Then, the optimal task insertion is iteratively selected for each path to maximize the overall task utility and improve rescue efficiency while satisfying constraint conditions. Algorithm 2 describes pseudocode for the utility-based initialization strategy.

The second heuristic scheme is the rescue cost-based initialization strategy. The rescue cost-based initialization strategy has the same structure as the utility-based initialization strategy, except that the utility value is replaced by the ranking of rescue cost. This strategy assigns tasks with low rescue cost to each robot as the starting point, selects the optimal task insertion for each path in turn, and constructs a multi-robot initial solution under the premise of satisfying the constraint conditions.

As a population-based metaheuristic algorithm, high-quality and diverse populations are essential for the evolution of the QPIG algorithm. Therefore, we use the proposed heuristic algorithms (utility-based initialization strategy and rescue cost-based initialization strategy) to generate two individuals to ensure the population's quality. Meanwhile, randomly generate the remaining *ps*-2 individuals to guarantee the diversity of the population.

Algorithm 2: Utility-based initialization strategy

Input: parameters
Output: the solution: π

$N' \leftarrow N$;
Calculate $f_{0j}, j \in \{1, 2, \ldots, n\}$, $n = sizeof(N')$;
Sort tasks in N' by ascending order of f_{0j}, and select the top m tasks;
Delete the above m tasks in N';
$k = 1, i \leftarrow \pi_{k,1}$; π
while $N' \neq \phi$ **do**
    Calculate f_{ij};
    $f_{\min} = min\{f_{i1}, f_{i2}, \ldots, f_{iN}\}$;
    Insert the task x of $f_{\min}$ into π_k;
    if the constraint is satisfied **then**
        $i \leftarrow x$, delete task x from N';
    else
        $k = k + 1, i \leftarrow \pi_{k,1}$;
    end
end
return π;

4.3. Destruction and Reconstruction Phase

The IG algorithm perturbs the current solution during the destruction and reconstruction phase, improving the diversity of the search process. In the destruction phase, l_{d} tasks are chosen randomly from the path sequence, yielding two subsequences: π^D, which includes the removed tasks, and π^R, which contains the remaining tasks. Parameter l_{d} represents the destruction size of the path sequence. The value of l_{d} is analyzed and set in Section 5.3. In the construction phase, the tasks π^D extracted from the destruction stage are reinserted into the sequence π^R. First, randomly extract a task δ from the sequence π^D. After attempting to insert task δ into all positions of π^R, select the position that minimizes the rescue cost for insertion. This process continues until all the removed tasks have been reinserted into the path sequence.

To enhance the diversity and quality of solutions, this study proposes a competition-oriented destruction-reconstruction mechanism. This mechanism includes a history-based exploration strategy, a collective intelligence-driven strategy, and an elite-oriented optimization strategy. Among them, the history-based exploration strategy destroys and reconstructs the random selection of individuals within the history solutions, enhancing the diversity of solutions; the collective intelligence-driven strategy performs destruction reconstruction operations on all individuals in the population while retaining the one with the best rescue cost and mining global information to enhance search depth; and the elite-oriented optimization strategy focuses on the current optimal individual and conducts fine optimization around their neighborhood to enhance local search ability. Moreover, this mechanism evaluates the candidate solutions generated by the three strategies based on their rescue cost and selects the best-performing solution. Algorithm 3 provides the pseudocode for the competition-oriented destruction-reconstruction mechanism.

Algorithm 3: Competition-oriented destruction-reconstruction mechanism

Input: population: *POP*, population size: *ps*, destruction-reconstruction size: l_d, the set of historical solutions: *Histsol*

Output: new population: *POP′*

Candsol ← ϕ, *Union* ← ϕ;
History-based exploration strategy # indicates a remark.
π_{rh} ← randomly select an individual from *Histsol*;
π_{rh}^D, π_{rh}^R ← *Destruction*(π_{rh}, l_d);
π^1 ← *Reconstruction*(π_{rh}^D, π_{rh}^R);
Add π^1 to *Candsol*;
Collective intelligence-driven strategy
for each individual ∈ *POP* **do**
    π_i^D, π_i^R ← *Destruction*(π_i, l_d);
    π'_i ← *Reconstruction*(π_i^D, π_i^R);
end
POP^1 ← update *POP*;
$idxbest_1 = arg(min_{j=}^{ps}({}_1f(POP^1{}_j))$;
$\pi^2 \leftarrow POP_{1idxbest}$;
Add π^2 to *Candsol*;
Elite-oriented optimization strategy
$idxbest_2 = arg(min_{j=1}^{ps}(f(POP_j))$;
$\pi_{eo} \leftarrow POP_{idxbest2}$;
π_{eo}^D, π_{eo}^R ← *Destruction*(π_{eo}, l_d);
π^3 ← *Reconstruction*(π_{eo}^D, π_{eo}^R);
Add π^3 to *Candsol*;
Union ← *POP* ∪ *Candsol*;
POP′ ← Select the top *ps* individuals with the rescue cost ranking from *Union*;
return *POP′*;

4.4. Q-Learning-Based Local Search Strategy

Q-learning is an RL algorithm that allows agents to identify the most effective actions through feedback obtained from reward mechanisms. It mainly consists of intelligent agents, environment, state, actions, and rewards. An agent takes actions in a given state, and the environment will transition to a new state based on the actions and the current state and reward the agent. Through repeated loops, the experience of the intelligent agent will be improved. The Q-table is an important component of Q-learning, used to guide individuals in selecting actions during different iteration processes, as shown in Table 2. Rows represent states, and columns represent actions. Each combination of states and actions corresponds to a Q-value, which reflects the learning process of the agent. The Q-value update is given by the following formula:

$$Q(S_t, a_t) = (1 - \alpha)Q(S_t, a_t) + \alpha(R + \gamma \max Q(S_{t+1}, a_{t+1})) \quad (9)$$

where $Q(S_t, a_t)$ denotes the Q-value for executing action a_t in state S_t, α is the learning rate, R represents the reward received after taking the action, γ is the discount rate, and $\max Q(S_{t+1}, a_{t+1})$ corresponds to the maximum expected Q-value in the Q-table for state S_{t+1}.

Table 2. Q-table.

	a_1	a_2	...	a_t
S_1	$Q(S_1, a_1)$	$Q(S_1, a_2)$	...	$Q(S_1, a_t)$
S_2	$Q(S_2, a_1)$	$Q(S_2, a_2)$	...	$Q(S_2, a_t)$
...	...	...	...	...
S_n	$Q(S_n, a_1)$	$Q(S_n, a_2)$	...	$Q(S_n, a_t)$

(1) Status

The solutions form the set of states in Q-learning, with each individual representing a particular state. Specifically, state S_t represents the current solution of the QPIG algorithm at time step t. The state S_t is observable and distinguishable, meaning that the agent can accurately identify and distinguish different states based on the characteristics of the current solution. State transitions satisfy the Markov property, meaning that future states depend only on the current state and the selected action and are independent of historical states.

(2) Action

The action indicates the neighborhood structure of the solution in the Q-learning environment model. Currently, no universal improvement strategy based on a neighborhood structure exists that is applicable to all problems. In QPIG, the set of neighborhood structures is designed with six operators specific to the problem. The critical path mentioned refers to the path sequence with the highest rescue cost. These neighborhood operators are explained as follows, and the graphical representation is provided in Figure 4.

(a) Critical path swap operator (*LS_op*1)

Identify the critical path first, and then randomly choose a task from it as the critical task, T_c. Then, exchange T_c with each non-critical task T_n until a promising path sequence is found, as shown in Figure 4a.

(b) Critical path and non-critical path swap operator (*LS_op*2)

Randomly choose a critical task T_c from the critical path, and randomly choose a non-critical path from the solution. Swap the critical task T_c with each non-critical task T_n in the non-critical path to generate a new path sequence, as depicted in Figure 4b.

(c) Critical path insertion operator (*LS_op*3)

Select a task, T_c, at random from the critical path. Subsequently, T_c is inserted into the preceding and following locations of every non-critical task T_n for swapping to generate a new path sequence, as illustrated in Figure 4c.

(d) Critical path and non-critical path insertion operator (*LS_op*4)

Choose a task T_c at random from the critical path, and randomly choose a non-critical path from the solution. Subsequently, insert the critical task T_c into the preceding and following locations of every non-critical task T_n in the non-critical path to generate a promising path sequence, as presented in Figure 4d.

(e) Path reverse operator (*LS_op*5)

Select a continuous task in the critical path or non-critical path, and reverse the subsequence to generate a promising path sequence, as depicted in Figure 4e.

(f) Path right shift operator (*LS_op*6)

Move the subsequence to the right in the critical path or non-critical path, move the last element of the subsequence to the first position of the subsequence, and move the other elements back in sequence to generate a new path sequence, as shown in Figure 4f.

(3) Action selection

The ε-greedy strategy is employed to explore new action options during the training process, thereby improving the global optimality of learning. In this study, a larger ε value is set in the initial stage to encourage the algorithm to explore a wide range of local operators. As iterations continue, the value of ε progressively decreases linearly, causing the strategy to lean towards utilizing existing knowledge. The formula used to compute ε is as follows:

$$\varepsilon_g = \max\left(\varepsilon_{\min}, \varepsilon_0 - \frac{g}{G}(\varepsilon_0 - \varepsilon_{\min})\right) \tag{10}$$

where ε_0 is the initial value, and $\varepsilon_{\min}$ is the minimum value of ε, g indicates the current iteration count, and G denotes the total allowed iterations.

(4) Reward

After executing an action, an individual can receive a reward R. In QPIG, the reward R is calculated as follows:

$$R = \begin{cases} 1 & f\prime < f \\ 0 & f\prime = f \\ -1 & f\prime > f \end{cases} \tag{11}$$

where f' denotes the total rescue cost of the new solution, and f represents the total rescue cost of the current solution.

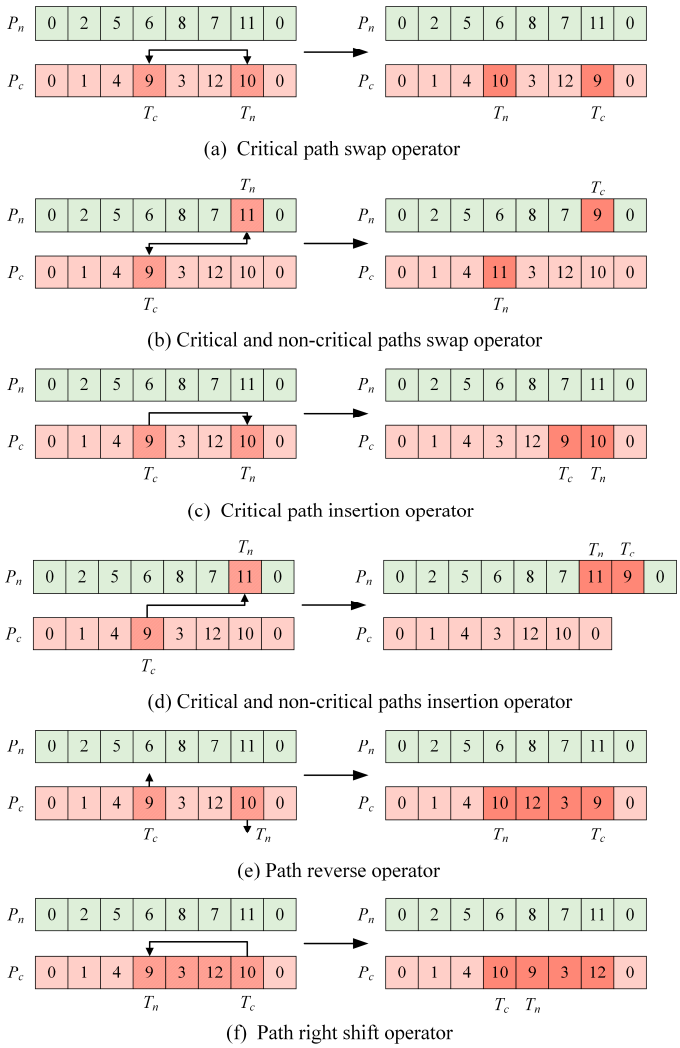


Figure 4. Graphical representation of six neighborhood operators.

4.5. Acceptance Criteria

The acceptance criterion based on simulated annealing (SA) is employed to enhance the diversity of solutions [40]. Its core strategy involves accepting the current solution with a given probability p if the rescue cost of the newly generated solution exceeds that of the current one, rather than directly replacing it with the new solution. The calculation formula for receiving probability p is as follows:

$$p = \min\left\{\exp\left(\frac{F(\pi) - F(\pi_{new})}{Temperature}\right), 1\right\} \tag{12}$$

where π represents the current solution and π_{new} denotes the newly generated solution. The temperature settings are as follows:

$$Temperature = \sigma \times \frac{\sum_{k=1}^{m}\sum_{i=1}^{n} t_{ik}}{10 \times m \times n} \tag{13}$$

where σ is the constant temperature value, t_{ik} is the service time of robot k at rescue point i, m denotes the number of robots, and n refers to the number of rescue points.

4.6. Historical Information-Based Constructive Strategy

The construction strategy based on historical information aims to generate a better path by borrowing high-quality parts from multiple historical solutions. Its core idea is to identify the parts that perform well by utilizing path data from historical solutions and optimize them accordingly. By integrating the advantages of different solutions, this strategy aims to reduce the search space, thereby improving the algorithm's convergence efficiency. The pseudocode of the historical information-based constructive strategy is shown in Algorithm 4. The historical path allocation operator is described in Algorithm 5. Algorithm 6 provides pseudocode for the path refinement operator.

Algorithm 4: Historical information-based constructive strategy

Input: the set of current solutions: *RSeq*
Output: historical solution: π_{history}

```
rmax = 0;
for each π ∈ RSeq do
    cn = count (unique robots in π);
    if cn > rmax then
        rmax = cn;
    end
end
RobMat = CreateRobotPathMatrix(rmax);
PE_best = Historical path allocation operator(rmax, RobMat);
π_history = Path refinement operator(PE_best, RSeq);
return π_history;
```

Algorithm 5: Historical path allocation operator

Input: maximum number of robots: *rmax*, path matrix: *RobMat*
Output: the optimal path for each robot: PE_{best}

```
nsol = size(RobMat, 2);
for i = 1:rmax do
    P_best = inf;
    for j = 1:nsol do
        P_new = RobMat(i, j);
        if f(P_new) < f(P_best) then
            P_best = P_new;
        end
    end
PE_best(i) = P_best;
end
return PE_best;
```

Algorithm 6: Path refinement operator

Input: the optimal path for each robot: PE_{best}, the set of current solutions: *RSeq*
Output: historical solution: π_{history}

```
π_opt = CombinePath(PE_best);
π_opt1 = RemoveDuplicates(π_opt);
π_ref = RSeq(randi([1, length(RSeq)]));
MN = FindMissingPoints(π_ref, π_opt1);
for each point z in MN do
    bestpos = FindOptimalInsertion(π_opt1, z);
    π_opt1 = InsertPointsToPath(π_opt1, z, bestpos);
end
π_history = π_opt1;
return π_history;
```

4.7. Computational Time Complexity of QPIG

In the initialization phase, two problem-specific heuristic strategies are designed, including the utility-based initialization strategy and the rescue cost-based initialization strategy. The time complexity of the utility-based initialization strategy is $O(n^2)$, and similarly, the time complexity of the rescue cost-based initialization strategy is also $O(n^2)$, where n represents the number of rescue points. In the destruction and reconstruction phase, a competition-oriented destruction-reconstruction mechanism is developed to improve the diversity of the search process. Specifically, the time complexity of the history-based exploration strategy is $O(l_{\text{d}})$, where l_{d} represents the destruction-reconstruction size. The collective intelligence-driven and elite-oriented optimization strategies each have a time complexity of $O(ps \cdot l_{\text{d}})$, where ps denotes the population size. Additionally, operations such as merging populations and selecting the top individuals introduce a complexity of $O(ps)$ and $O(ps\log ps)$, respectively. Therefore, the overall time complexity of this mechanism is $O(ps \cdot l_{\text{d}} + ps\log ps)$. In the local search phase, six neighborhood operators, i.e., critical path swap, critical path and non-critical path swap, critical path insertion, critical path and non-critical path insertion, path reverse, and path right shift operators, each have a time complexity of $O(n)$. Additionally, operations such as Q-value updates, reward calculations, and state management are also performed. Therefore, the overall time complexity of the Q-learning-based local search strategy is $O(n \cdot G)$, where G is the number of iterations in this

strategy. The time complexity of the historical information-based constructive strategy is $O(n^2)$. In summary, the time complexity of the QPIG algorithm is $O(n^2)$.

5. Simulation Results and Analysis

5.1. Experimental Instances

To evaluate the effectiveness of the QPIG algorithm, we conducted experiments in MATLAB R2020a on an Intel Core i7 2.10 GHz processor and 16 GB RAM. The experiment selected geographic information from Yantai, a city in Shandong Province, as the actual background. The test data were sourced from the Solomon dataset [41]. The Solomon dataset contains three types of instances, namely random instances, clustered instances, and semi-clustered instances. The extended instance is consistent with the Solomon dataset in terms of coordinates, demand, service time, and time window, and it generates initial task utility randomly for each rescue point. To distinguish these extended instances, we added a prefix of "T" to the instance names, indicating that these instances are based on the Solomon dataset and have added task utility to the extended versions. Taking TC101 as an example, the distribution information of rescue centers and rescue points is shown in Figure 5. The red cross in the picture denotes the rescue center, and the red dots represent the rescue points. To guarantee fairness, identical stopping criteria and a consistent maximum CPU time of 100 s are applied to all algorithms. The effectiveness of the algorithms is evaluated by taking the average of 30 independent runs for each instance.

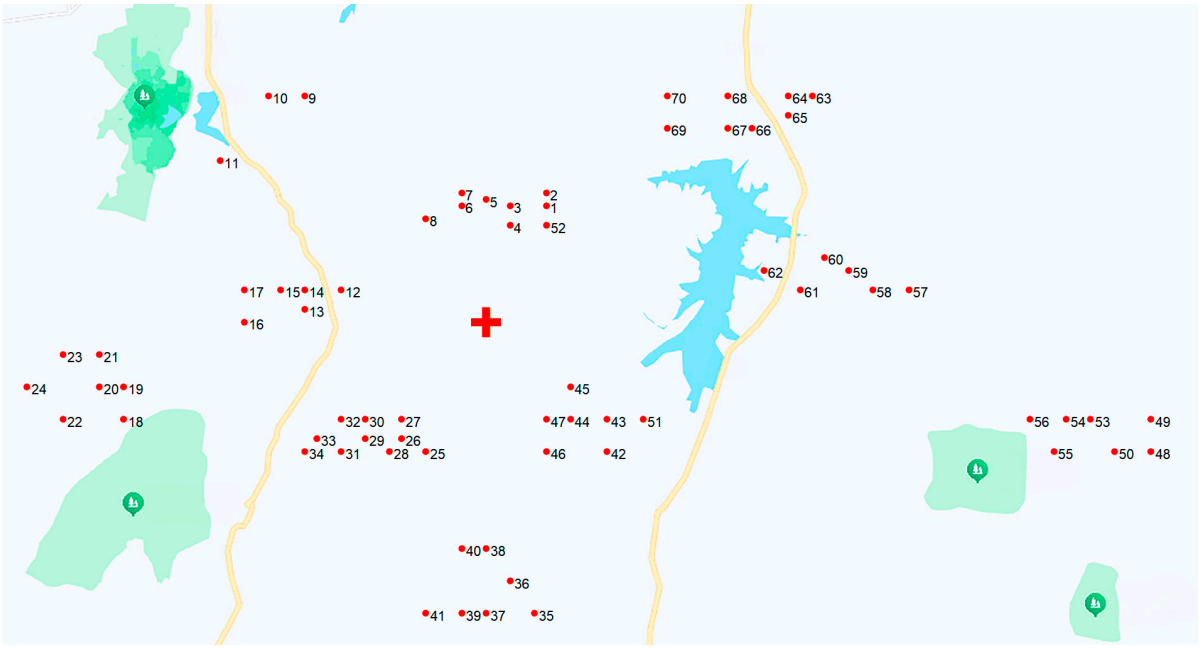


Figure 5. Locations of the rescue center and rescue points in instance TC101.

5.2. Weight Coefficient Setting

The metrics in the rescue task differ in their ranges of variation. After preliminary experiments, all metrics are normalized using min–max normalization to ensure comparability. The normalization equation is expressed as follows:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{14}$$

where x and x_{norm} denote the original and normalized values, respectively, and $x_{\min}$ and $x_{\max}$ are the minimum and maximum values of the corresponding metric. Additionally,

the weights are determined using a least squares fitting approach based on reference solutions [42], thereby reflecting the relative importance of each indicator. The weight coefficients are computed as follows: $w_l = 0.42$, $w_n = 0.62$, $w_t = 0.09$, and $w_r = 0.12$.

5.3. Parameter Settings and Sensitivity Analysis

The algorithm's performance is greatly influenced by the adjustment of the appropriate parameters. In this work, four parameters are considered, including population size (ps), the length of removed tasks in the destruction and reconstruction phase (l_d), learning rate (α), and discount rate (γ). Based on our preliminary experiments on instance TC101, the parameter levels are presented in Table 3. Similarly to [43,44], the design-of-experiment (DOE) is employed to assess how these four parameters influence the performance of the algorithm. The parameter combinations are determined using the orthogonal array L_{16} (4^4), as shown in Table 4. To ensure statistical reliability, the QPIG runs each parameter combination independently 30 times and collects the average rescue cost as the response variable (RV). Figure 6 illustrates the trend of the factor levels for the parameters. Moreover, Table 5 provides the average response value and ranking of each parameter. It can be observed that (1) these four parameters have a significant impact on the performance of the proposed QPIG. (2) The parameter l_d exerts the most significant influence on the algorithm, followed by ps, γ, and α. (3) The parameter combination settings of $ps = 20$, $l_d = 10$, $\alpha = 0.3$, and $\gamma = 0.7$ can achieve optimal performance of the algorithm.

Table 3. Tested values for each parameter level.

Parameters	Parameter Level			
	1	**2**	**3**	**4**
ps	15	20	30	50
l_d	5	10	15	20
α	0.1	0.2	0.3	0.4
γ	0.6	0.7	0.8	0.9

Table 4. Orthogonal array and RV values.

Combination	Parameters				Response Variable
	ps	l_d	α	γ	
1	15	5	0.1	0.6	1716.09
2	15	10	0.2	0.7	1654.36
3	15	15	0.3	0.8	1634.45
4	15	20	0.4	0.9	1670.21
5	20	5	0.2	0.8	1700.05
6	20	10	0.1	0.9	1630.80
7	20	15	0.4	0.6	1675.06
8	20	20	0.3	0.7	1662.97
9	30	5	0.3	0.9	1723.61
10	30	10	0.4	0.8	1690.11
11	30	15	0.1	0.7	1671.90
12	30	20	0.2	0.6	1699.16
13	50	5	0.4	0.7	1685.63
14	50	10	0.3	0.6	1679.17
15	50	15	0.2	0.9	1692.32
16	50	20	0.1	0.8	1711.55

Table 5. The average RPI response values and the rank of each parameter.

Level	ps	l_{d}	α	γ
1	1669	1706	1683	1692
2	1667	1664	1686	1669
3	1696	1668	1675	1684
4	1692	1686	1680	1679
Delta	29	43	11	24
Rank	2	1	4	3

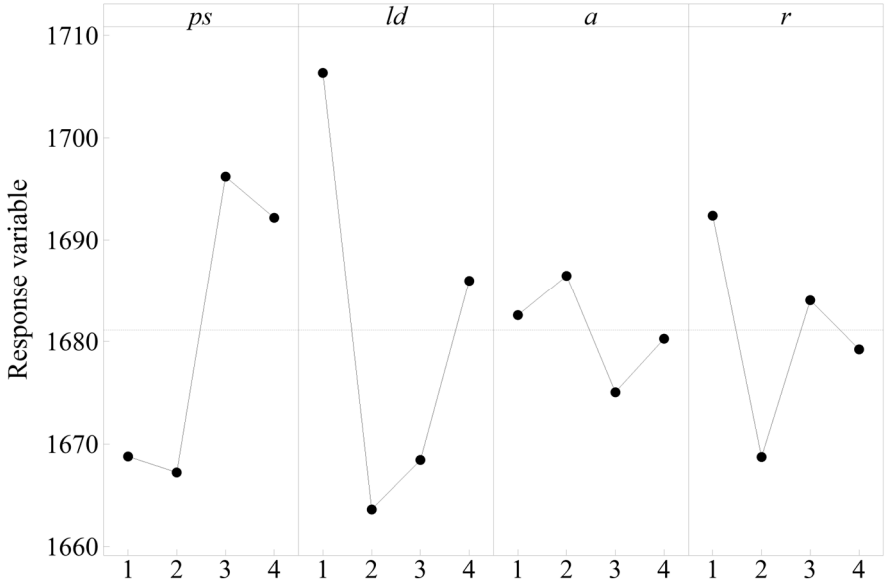


Figure 6. Factor level trend of parameters.

5.4. Performance of the Developed Components

This study presents three key components: two problem-specific heuristic schemes, the competition-oriented destruction-reconstruction mechanism, and the historical information-based constructive strategy. To evaluate the performance of the three proposed components, we investigated four algorithms: the first is the QPIG, and the second does not use two problem-specific heuristic schemes (QPIG-NI). Indeed, QPIG-NI employs a random approach to establish the initial population, while all other components stay the same. Additionally, the third is not using the competition-oriented destruction-reconstruction mechanism (QPIG-ND), and the fourth is not using the historical information-based constructive strategy (QPIG-NH). All algorithms are independently run 30 times on 56 instances. The algorithm's performance is quantified using the relative percentage increase (RPI).

$$RPI = \frac{f_{\mathrm{c}} - f_{\mathrm{best}}}{f_{\mathrm{best}}} \times 100 \tag{15}$$

where f_{best} is the optimal value found by all compared algorithms, and f_{c} is the optimal value from a specific algorithm.

Table 6 lists the data of all compared algorithms. The first column is the name of the instances, the second column represents the optimal solutions of the four algorithms, the next four columns denote the experimental results of the QPIG, QPIG-NI, QPIG-ND, and QPIG-NH algorithms, and the RPI values correspond to the four algorithms. The comparison results indicate that (1) among the 56 large-scale instances, the QPIG algorithm obtained 49 optimal solutions, whereas the other three algorithms achieved only 7 optimal solutions. Specifically, (a) in the TC and TR2 instance series, QPIG obtained all optimal

solutions. In these test instances, the rescue point distribution in the TC series is clustered, while in the TR2 series, it is randomly distributed with relatively wide time windows. (b) In the TR1 instance series, the QPIG algorithm obtained 10 optimal solutions, while QPIG-NI obtained two optimal solutions. In the TR1 series, the rescue point distribution is random, and the time windows are relatively narrow. The reason for this situation is as follows: Although heuristic initialization accelerates convergence, it may lead to local optima. Random initialization allows the algorithm to start from different initial solutions, enabling a broader exploration of the solution space and avoiding premature convergence to local optima. In the case of randomly distributed rescue points and narrow time windows, heuristic initialization may favor certain specific distributions, limiting the exploration of the solution space. Random initialization avoids this problem and provides more opportunities for exploration. (c) In the TRC instance series, the QPIG algorithm obtained 12 optimal solutions, QPIG-NI obtained three optimal solutions, and QPIG-NH obtained two optimal solutions. In the TRC series, the rescue point distribution exhibits a mixed pattern of random and clustered distributions. The reasons for this situation are as follows: excessive reliance on historical information may cause the algorithm to be unable to adapt to new environments, especially in the case of a mixed distribution of rescue points that are both random and clustered. QPIG-NH can dynamically adjust the search strategy based on the current task and environment, better adapting to changing conditions. In mixed distribution scenarios, QPIG-NI starts from multiple random solutions, avoiding the limitations of heuristic initialization, enabling a broader exploration of the solution space, and preventing local optima. (2) The QPIG algorithm achieves an average RPI of 0.15, which is markedly lower than the mean RPI values achieved by the other three algorithms. The average RPI value of QPIG-ND is the highest, indicating that it has the most significant impact on the algorithm's performance. These data suggest that future improvements should focus on enhancing the destruction-reconstruction component to improve the algorithm's global search capability.

Table 6. Comparison results for QPIG, QPIG-NI, QPIG-ND, and QPIG-NH.

Instance	Best Value	Algorithms				RPI			
		QPIG	QPIG-NI	QPIG-ND	QPIG-NH	QPIG	QPIG-NI	QPIG-ND	QPIG-NH
TC101	1609.70	**1609.70**	1645.17	1774.19	1669.22	**0.00**	2.20	10.22	3.70
TC102	1726.05	**1726.05**	1746.08	1836.37	1738.15	**0.00**	1.16	6.39	0.70
TC103	1665.16	**1665.16**	1671.13	1758.55	1666.70	**0.00**	0.36	5.61	0.09
TC104	1590.13	**1590.13**	1599.36	1716.51	1603.89	**0.00**	0.58	7.95	0.87
TC105	1649.47	**1649.47**	1680.00	1773.87	1670.68	**0.00**	1.85	7.54	1.29
TC106	1595.79	**1595.79**	1621.46	1642.68	1600.00	**0.00**	1.61	2.94	0.26
TC107	1604.30	**1604.30**	1615.43	1749.23	1613.98	**0.00**	0.69	9.03	0.60
TC108	1561.28	**1561.28**	1580.57	1667.09	1567.56	**0.00**	1.24	6.78	0.40
TC109	1520.16	**1520.16**	1544.04	1616.34	1526.73	**0.00**	1.57	6.33	0.43
TC201	2823.61	**2823.61**	2836.77	2921.51	2839.42	**0.00**	0.47	3.47	0.56
TC202	2782.36	**2782.36**	2788.31	2862.43	2783.06	**0.00**	0.21	2.88	0.03
TC203	2779.81	**2779.81**	2794.54	2838.33	2780.00	**0.00**	0.53	2.11	0.01
TC204	2797.82	**2797.82**	2836.66	2860.28	2815.44	**0.00**	1.39	2.23	0.63
TC205	2804.19	**2804.19**	2815.23	2860.57	2807.40	**0.00**	0.39	2.01	0.11
TC206	2608.94	**2608.94**	2659.28	2691.27	2614.82	**0.00**	1.93	3.16	0.23
TC207	2619.96	**2619.96**	2650.60	2724.46	2623.79	**0.00**	1.17	3.99	0.15
TC208	2612.20	**2612.20**	2639.99	2696.85	2616.07	**0.00**	1.06	3.24	0.15
TR101	1261.10	**1261.10**	1262.20	1338.54	1271.44	**0.00**	0.09	6.14	0.82
TR102	1129.82	**1129.82**	1135.80	1215.47	1138.39	**0.00**	0.53	7.58	0.76

Table 6. *Cont.*

Instance	Best Value	Algorithms				RPI			
		QPIG	QPIG-NI	QPIG-ND	QPIG-NH	QPIG	QPIG-NI	QPIG-ND	QPIG-NH
TR103	1096.06	**1096.06**	1097.41	1167.82	1108.18	**0.00**	0.12	6.55	1.11
TR104	1008.83	**1008.83**	1010.86	1111.04	1019.06	**0.00**	0.20	10.13	1.01
TR105	1129.40	**1129.40**	1133.46	1209.42	1142.65	**0.00**	0.36	7.09	1.17
TR106	1108.09	**1108.09**	1110.52	1185.31	1120.74	**0.00**	0.22	6.97	1.14
TR107	1050.06	1052.40	**1050.06**	1140.52	1068.60	0.22	**0.00**	8.61	1.77
TR108	972.06	**972.06**	973.86	1089.26	974.60	**0.00**	0.19	12.06	0.26
TR109	1021.67	**1021.67**	1023.39	1134.18	1024.08	**0.00**	0.17	11.01	0.24
TR110	1031.55	**1031.55**	1037.60	1153.14	1049.50	**0.00**	0.59	11.79	1.74
TR111	1035.51	**1035.51**	1037.11	1126.43	1044.26	**0.00**	0.16	8.78	0.85
TR112	960.91	962.40	**960.91**	1107.71	968.27	0.15	**0.00**	15.28	0.77
TR201	1467.80	**1467.80**	1472.24	1561.33	1478.62	**0.00**	0.30	6.37	0.74
TR202	1515.16	**1515.16**	1521.19	1619.63	1526.68	**0.00**	0.40	6.89	0.76
TR203	1426.66	**1426.66**	1430.03	1515.00	1437.62	**0.00**	0.24	6.19	0.77
TR204	1377.51	**1377.51**	1378.42	1471.82	1407.19	**0.00**	0.07	6.85	2.15
TR205	1460.35	**1460.35**	1462.50	1549.42	1466.99	**0.00**	0.15	6.10	0.46
TR206	1425.37	**1425.37**	1433.72	1518.82	1437.53	**0.00**	0.59	6.56	0.85
TR207	1337.63	**1337.63**	1338.79	1416.76	1344.20	**0.00**	0.09	5.92	0.49
TR208	1285.34	**1285.34**	1287.70	1352.25	1293.81	**0.00**	0.18	5.21	0.66
TR209	1306.99	**1306.99**	1310.11	1376.45	1309.29	**0.00**	0.24	5.31	0.18
TR210	1449.77	**1449.77**	1458.20	1556.45	1460.37	**0.00**	0.58	7.36	0.73
TR211	1196.74	**1196.74**	1216.65	1263.81	1200.07	**0.00**	1.66	5.60	0.28
TRC101	1169.57	**1169.57**	1180.21	1331.81	1190.16	**0.00**	0.91	13.87	1.76
TRC102	1159.09	**1159.09**	1165.51	1294.60	1173.73	**0.00**	0.55	11.69	1.26
TRC103	1141.06	**1141.06**	1145.89	1152.81	1147.95	**0.00**	0.42	1.03	0.60
TRC104	1040.57	**1040.57**	1044.43	1302.51	1171.26	**0.00**	0.37	25.17	12.56
TRC105	1163.66	**1163.66**	1163.83	1214.35	1190.34	**0.00**	0.01	4.36	2.29
TRC106	1054.76	1059.40	**1054.76**	1253.97	1076.29	0.44	**0.00**	18.89	2.04
TRC107	1026.93	1067.67	1073.58	1204.33	**1026.93**	3.97	4.54	17.27	**0.00**
TRC108	1012.76	**1012.76**	1013.68	1204.33	1026.93	**0.00**	0.09	18.92	1.40
TRC201	1559.46	**1559.46**	1567.41	1712.76	1574.04	**0.00**	0.51	9.83	0.93
TRC202	1507.01	1554.15	1547.12	1624.32	**1507.01**	3.13	2.66	7.78	**0.00**
TRC203	1497.82	**1497.82**	1508.79	1646.90	1518.81	**0.00**	0.73	9.95	1.40
TRC204	1482.91	**1482.91**	1491.86	1756.87	1618.00	**0.00**	0.60	18.47	9.11
TRC205	1586.29	1587.36	**1586.29**	1605.35	1635.51	0.07	**0.00**	1.20	3.10
TRC206	1435.68	1445.01	**1435.68**	1531.42	1476.00	0.65	**0.00**	6.67	2.81
TRC207	1343.05	**1343.05**	1343.83	1402.19	1398.73	**0.00**	0.06	4.40	4.15
TRC208	1181.76	**1181.76**	1200.11	1402.19	1217.19	**0.00**	1.55	18.65	3.00
Mean	1513.71	**1515.62**	1524.83	1621.64	1531.75	**0.15**	0.69	8.11	1.36

The bold font indicates the minimum values.

To determine the differences between these compared algorithms, we use the analysis of variance (ANOVA) method. Figure 7 illustrates the ANOVA results of these four algorithms. Figure 7a displays the results of the ANOVA implemented by QPIG and QPIG-NI. The p-value is 4.76×10^{-10}, substantially lower than 0.05, indicating a notable difference between these two algorithms and verifying the effectiveness of the two problem-specific heuristic schemes. The ANOVA between QPIG and QPIG-ND shows a p-value of 1.32×10^{-21}, which is much smaller than 0.05, as illustrated in Figure 7b. This indicates that the competition-oriented destruction-reconstruction mechanism is effective. The p-value is 6.94×10^{-5}, which is much lower than 0.05, demonstrating a notable difference between QPIG and QPIG-NH and verifying the effectiveness of the historical information-based constructive strategy, as illustrated in Figure 7c. Figure 7d presents the result plot between QPIG components obtained through Tukey's test within a 95% LSD confidence interval. The data indicate that the competition-oriented destruction-reconstruction mechanism is the most important component of QPIG, followed by the historical information-based constructive strategy and two problem-specific heuristic schemes, with their importance decreasing in order.

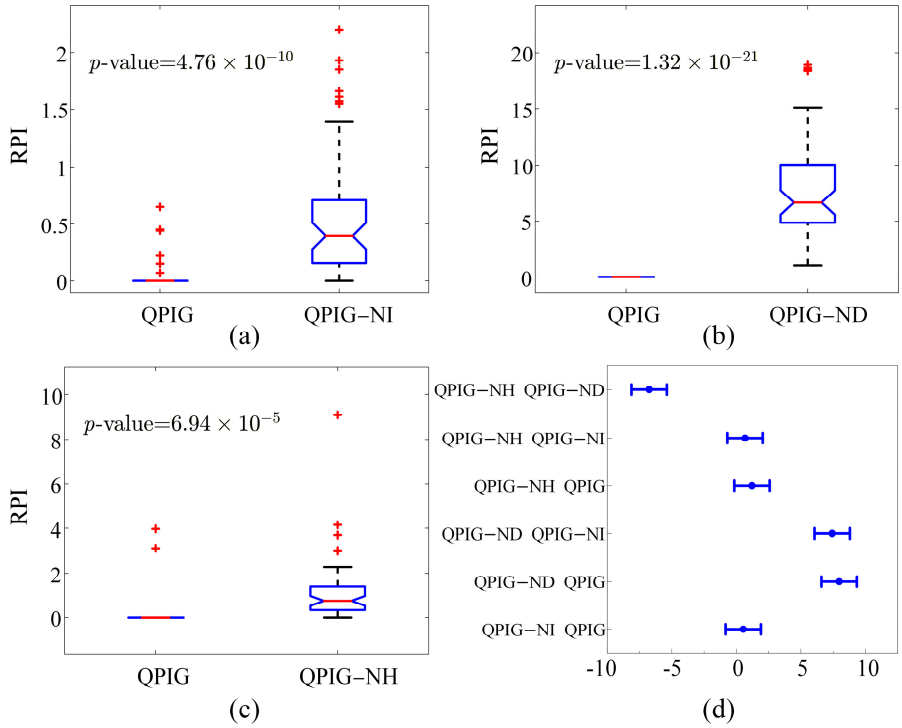


Figure 7. ANOVA results of the compared algorithms. (**a**) ANOVA of QPIG and QPIG-NI; (**b**) ANOVA of QPIG and QPIG-ND; (**c**) ANOVA of QPIG and QPIG-NH; (**d**) Tukey test 95% LSD intervals among QPIG components.

To illustrate the impact of the Q-learning-based local search strategy, we designed four algorithms: the first is QPIG, the second does not use a local search strategy (QPIG-NL), the third does not use Q-learning and instead employs a random method to select six neighborhood operators (QPIG-RS), and the fourth also does not use Q-learning and instead employs a probabilistic method to select six neighborhood operators (QPIG-PS). Independently run 30 experiments to collect data obtained by all algorithms. Table 7 presents the comparative results of QPIG, QPIG-NL, QPIG-RS, and QPIG-PS algorithms. It can be inferred from Table 7 that (1) among the given 56 examples, the QPIG algorithm obtained all optimal solutions; (2) the mean RPI obtained by the QPIG algorithm is 0, which is much lower than the other three algorithms. Figure 8 shows the ANOVA results for the QPIG, QPIG-NL, QPIG-RS, and QPIG-PS algorithms. The p-value is 1.24×10^{-22}, significantly lower than 0.05, demonstrating the effectiveness of Q-Learning in selecting local search operators. This significant improvement is due to the adaptive nature of Q-learning, which dynamically adjusts the local search strategy based on the algorithm's performance history. Compared to random or probabilistic methods, Q-learning can focus on more promising search directions, thereby enhancing the algorithm's local search ability.

Table 7. Comparison results for QPIG, QPIG-NL, QPIG-RS and QPIG-PS.

Instance	Best Value	Algorithms				RPI			
		QPIG	QPIG-NL	QPIG-RS	QPIG-PS	QPIG	QPIG-NL	QPIG-RS	QPIG-PS
TC101	1609.70	**1609.70**	1644.04	1709.19	1711.37	**0.00**	2.13	6.18	6.32
TC102	1726.05	**1726.05**	1765.66	1822.97	1817.18	**0.00**	2.29	5.62	5.28
TC103	1665.16	**1665.16**	1799.51	1876.54	1809.02	**0.00**	8.07	12.69	8.64
TC104	1590.13	**1590.13**	1823.85	1976.83	1858.45	**0.00**	14.70	24.32	16.87
TC105	1649.47	**1649.47**	1717.10	1861.16	1769.01	**0.00**	4.10	12.83	7.25
TC106	1595.79	**1595.79**	1714.59	1745.00	1777.13	**0.00**	7.44	9.35	11.36
TC107	1604.30	**1604.30**	1751.81	1775.09	1798.98	**0.00**	9.19	10.65	12.13
TC108	1561.28	**1561.28**	1782.14	1785.05	1790.64	**0.00**	14.15	14.33	14.69
TC109	1520.16	**1520.16**	1762.69	1738.91	1750.02	**0.00**	15.95	14.39	15.12
TC201	2823.61	**2823.61**	2851.75	2885.25	2900.56	**0.00**	1.00	2.18	2.73
TC202	2782.36	**2782.36**	2829.74	2854.45	2864.94	**0.00**	1.70	2.59	2.97
TC203	2779.81	**2779.81**	2834.61	2878.16	2849.53	**0.00**	1.97	3.54	2.51
TC204	2797.82	**2797.82**	2865.15	2913.52	2879.87	**0.00**	2.41	4.14	2.93
TC205	2804.19	**2804.19**	2829.98	2885.01	2859.39	**0.00**	0.92	2.88	1.97
TC206	2608.94	**2608.94**	2647.00	2665.41	2681.70	**0.00**	1.46	2.16	2.79
TC207	2619.96	**2619.96**	2667.47	2681.99	2718.80	**0.00**	1.81	2.37	3.77
TC208	2612.20	**2612.20**	2651.36	2672.35	2674.99	**0.00**	1.50	2.30	2.40
TR101	1261.10	**1261.10**	1265.36	1279.88	1293.19	**0.00**	0.34	1.49	2.54
TR102	1129.82	**1129.82**	1144.20	1152.49	1159.71	**0.00**	1.27	2.01	2.65
TR103	1096.06	**1096.06**	1107.22	1191.67	1124.81	**0.00**	1.02	8.72	2.62
TR104	1008.83	**1008.83**	1034.86	1162.30	1059.19	**0.00**	2.58	15.21	4.99
TR105	1129.40	**1129.40**	1139.54	1211.67	1165.21	**0.00**	0.90	7.29	3.17
TR106	1108.09	**1108.09**	1116.46	1131.47	1142.00	**0.00**	0.75	2.11	3.06
TR107	1052.40	**1052.40**	1065.23	1078.72	1095.00	**0.00**	1.22	2.50	4.05
TR108	972.06	**972.06**	982.93	1006.20	1022.77	**0.00**	1.12	3.51	5.22
TR109	1021.67	**1021.67**	1031.34	1051.54	1024.25	**0.00**	0.95	2.92	0.25
TR110	1031.55	**1031.55**	1049.17	1080.21	1094.20	**0.00**	1.71	4.72	6.07
TR111	1035.51	**1035.51**	1045.57	1058.79	1076.57	**0.00**	0.97	2.25	3.97
TR112	962.40	**962.40**	979.39	1002.32	1033.17	**0.00**	1.77	4.15	7.35
TR201	1467.80	**1467.80**	1512.93	1552.74	1585.21	**0.00**	3.07	5.79	8.00
TR202	1515.16	**1515.16**	1565.63	1582.00	1621.33	**0.00**	3.33	4.41	7.01
TR203	1426.66	**1426.66**	1460.21	1613.33	1490.95	**0.00**	2.35	13.08	4.51
TR204	1377.51	**1377.51**	1409.67	1519.02	1431.62	**0.00**	2.33	10.27	3.93
TR205	1460.35	**1460.35**	1515.56	1609.91	1537.57	**0.00**	3.78	10.24	5.29
TR206	1425.37	**1425.37**	1474.07	1496.83	1513.72	**0.00**	3.42	5.01	6.20
TR207	1337.63	**1337.63**	1362.55	1395.94	1403.83	**0.00**	1.86	4.36	4.95
TR208	1285.34	**1285.34**	1307.34	1333.17	1328.91	**0.00**	1.71	3.72	3.39
TR209	1306.99	**1306.99**	1319.29	1342.50	1363.21	**0.00**	0.94	2.72	4.30
TR210	1449.77	**1449.77**	1471.81	1499.34	1505.00	**0.00**	1.52	3.42	3.81
TR211	1196.74	**1196.74**	1218.24	1238.91	1251.96	**0.00**	1.80	3.52	4.61
TRC101	1169.57	**1169.57**	1185.76	1217.83	1227.30	**0.00**	1.38	4.13	4.94
TRC102	1159.09	**1159.09**	1182.30	1202.53	1204.94	**0.00**	2.00	3.75	3.96
TRC103	1141.06	**1141.06**	1177.50	1353.37	1201.76	**0.00**	3.19	18.61	5.32
TRC104	1040.57	**1040.57**	1054.51	1227.28	1078.06	**0.00**	1.34	17.94	3.60
TRC105	1163.66	**1163.66**	1166.80	1244.01	1204.98	**0.00**	0.27	6.90	3.55
TRC106	1059.40	**1059.40**	1070.43	1105.15	1116.09	**0.00**	1.04	4.32	5.35
TRC107	1067.67	**1067.67**	1101.57	1135.60	1142.37	**0.00**	3.17	6.36	7.00
TRC108	1012.76	**1012.76**	1045.70	1067.90	1108.69	**0.00**	3.25	5.44	9.47
TRC201	1559.46	**1559.46**	1617.68	1671.01	1678.22	**0.00**	3.73	7.15	7.61
TRC202	1554.15	**1554.15**	1593.70	1674.47	1645.89	**0.00**	2.55	7.74	5.90
TRC203	1497.82	**1497.82**	1557.29	1770.06	1576.40	**0.00**	3.97	18.18	5.25
TRC204	1482.91	**1482.91**	1522.99	1739.00	1570.26	**0.00**	2.70	17.27	5.89
TRC205	1587.36	**1587.36**	1647.44	1775.65	1693.98	**0.00**	3.78	11.86	6.72
TRC206	1445.01	**1445.01**	1473.30	1519.19	1547.31	**0.00**	1.96	5.13	7.08
TRC207	1343.05	**1343.05**	1397.26	1427.21	1437.16	**0.00**	4.04	6.27	7.01
TRC208	1181.76	**1181.76**	1226.22	1273.42	1285.61	**0.00**	3.76	7.76	8.79
Mean	1515.62	**1515.62**	1563.17	1620.03	1599.18	**0.00**	3.10	7.23	5.70

The bold font indicates the minimum values.

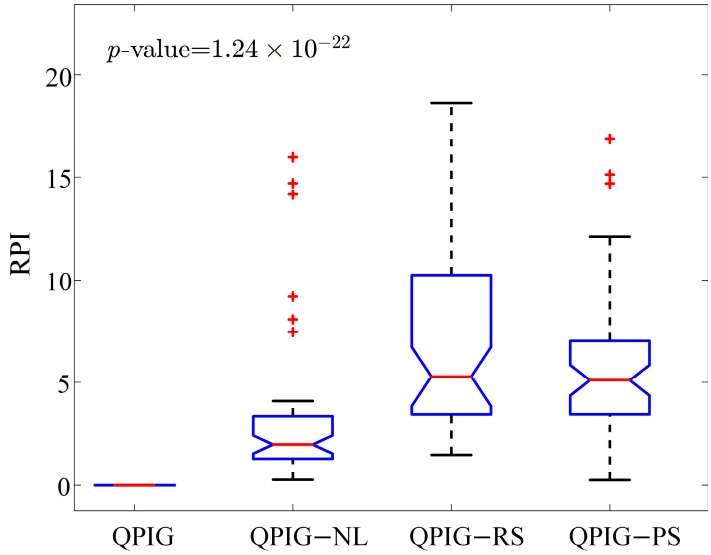


Figure 8. ANOVA comparisons of the QPIG, QPIG-NL, QPIG-RS, and QPIG-PS.

5.5. Comparison with Five Effective Algorithms

To analyze the performance of the QPIG algorithm in solving the MRRPP problem, five effective algorithms are selected for comparison, namely IIG [8], CDABC [45], MPSO [46], Q_DPIG [28], and QIG [47]. The IIG algorithm designs the feasible-first destruction-construction strategy aimed at broadening the search scope, which is used to solve the helicopter rescue routing problem with time windows and limited survival time. The CDABC algorithm has strong collaborative search capabilities and incorporates single-robot and multi-robot neighborhood operators. It utilizes dynamic strategy lists and the collaborative following-bee strategy to effectively escape local optima. The MPSO algorithm initializes the particle swarm within a predefined search space and iteratively updates to find the global optimal position, determining the key waypoints of the robot's path, with strong adaptability. The Q_DPIG algorithm uses Q-learning to select the appropriate destruction operator. An efficient and simple method is designed for the local optimization of the critical robot or the robot with the highest load, improving the stability of the solution. The QIG algorithm proposes a new selection mechanism inspired by Q-learning to provide strategic guidance, exhibiting excellent performance in convergence speed and solution quality. All compared algorithm parameters are calibrated, as shown in Table 8. We encoded and tested these five algorithms on the same computer, with consistent testing conditions. For each instance, each algorithm is executed 30 times, with a maximum CPU time of 100 s per execution, and the average rescue cost is recorded.

Table 8. The parameters for the compared algorithms.

Algorithms	Parameters
IIG	$l_d = 0.2$, $T = 0.4$
CDABC	$ps = 130$, $r_{cal} = 0$, $\lambda = 50$
MPSO	$ps = 100$, $\omega_{max} = 0.95$, $\omega_{min} = 0.4$, $c_{1max} = 2.0$, $c_{1min} = 0.5$, $c_{2max} = 2.0$, $c_{2min} = 0.5$, $v = [2,8]$
Q_DPIG	$ps = 15$, $l_d = 13$, $\alpha = 0.6$, $\gamma = 0.2$, $\in = 0.7$
QIG	$\alpha = 0.6$, $\gamma = 0.8$, $l_d = 3$

T: temperature parameter for acceptance criterion; r_{cal}: the experimentally calibrated parameter in the collaborative employed bee stage; λ: maximum iteration threshold for continuous unimproved individuals; ω_{max}: maximum inertia weight; ω_{min}: minimum

inertia weight; c_{1max}: maximum cognitive parameter; c_{1min}: minimum cognitive parameter; c_{2max}: maximum social parameter; c_{2min}: minimum social parameter; v: particle's velocity; $\in$: greedy selection probability.

Table 9 presents a comparison of the average rescue cost and RPI values across all algorithms. The following points are evident from Table 9: (1) the proposed QPIG achieved 51 optimal values in 56 instances; (2) the remaining five algorithms only obtained a total of 5 optimal solutions, among which the IIG algorithm obtained 4 optimal solutions, the QIG algorithm obtained one optimal solution, and the CDABC, MPSO, and Q_DPIG algorithms did not obtain any optimal solutions. (3) According to the last row of Table 9, the QPIG algorithm has an average RPI of 0.05, which is equivalent to 0.63%, 0.59%, 0.75%, 0.63%, and 1.05% of the average RPI of IIG, CDABC, MPSO, Q_DPIG, and QIG algorithms, respectively. The ANOVA shows a p-value of 1.29×10^{-25}, as shown in Figure 9, which is substantially lower than 0.05, demonstrating notable differences between the proposed QPIG algorithm and the algorithms used for comparison. The data distribution of the QPIG algorithm is relatively concentrated, with relatively few outliers, and its performance is relatively stable. This suggests that the QPIG algorithm outperforms others in solving the MRRPP problem.

Table 9. Comparison performance of QPIG with five algorithms.

Instance	Best Values	Algorithms						RPI					
		QPIG	IIG	CDABC	MPSO	Q_DPIG	QIG	QPIG	IIG	CDABC	MPSO	Q_DPIG	QIG
TC101	1609.70	**1609.70**	1652.79	1704.47	1631.47	1719.96	1656.36	**0.00**	2.68	5.89	1.35	6.85	2.90
TC102	1726.05	**1726.05**	1770.02	1919.69	1802.28	1833.75	1746.71	**0.00**	2.55	11.22	4.42	6.24	1.20
TC103	1665.16	**1665.16**	1824.25	1893.21	1811.05	1850.56	1740.90	**0.00**	9.55	13.70	8.76	11.13	4.55
TC104	1590.13	**1590.13**	1910.70	1879.14	1852.29	1903.80	1706.77	**0.00**	20.16	18.18	16.49	19.73	7.34
TC105	1649.47	**1649.47**	1791.49	1811.98	1800.37	1801.02	1752.36	**0.00**	8.61	9.85	9.15	9.19	6.24
TC106	1595.79	**1595.79**	1817.57	1797.75	1782.39	1796.05	1723.97	**0.00**	13.90	12.66	11.69	12.55	8.03
TC107	1604.30	**1604.30**	1833.31	1819.17	1824.62	1837.09	1758.92	**0.00**	14.27	13.39	13.73	14.51	9.64
TC108	1561.28	**1561.28**	1872.65	1838.50	1826.79	1839.07	1711.75	**0.00**	19.94	17.76	17.01	17.79	9.64
TC109	1520.16	**1520.16**	1899.60	1767.68	1816.86	1832.60	1733.87	**0.00**	24.96	16.28	19.52	20.55	14.06
TC201	2823.61	**2823.61**	3023.53	2990.16	3007.04	2959.29	2970.72	**0.00**	7.08	5.90	6.50	4.81	5.21
TC202	2782.36	**2782.36**	3015.04	2969.01	2985.18	2912.95	2931.22	**0.00**	8.36	6.71	7.29	4.69	5.35
TC203	2779.81	**2779.81**	3023.35	2987.33	2977.88	2893.12	2933.04	**0.00**	8.76	7.47	7.13	4.08	5.51
TC204	2797.82	**2797.82**	3059.34	3030.75	3025.89	2927.20	2945.83	**0.00**	9.35	8.33	8.15	4.62	5.29
TC205	2804.19	**2804.19**	3025.36	3003.30	3010.00	2914.38	2956.37	**0.00**	7.89	7.10	7.34	3.93	5.43
TC206	2608.94	**2608.94**	2893.50	2900.00	2887.10	2710.70	2789.23	**0.00**	10.91	11.16	10.66	3.90	6.91
TC207	2619.96	**2619.96**	2847.94	2881.26	2863.22	2730.10	2788.66	**0.00**	8.70	9.97	9.28	4.20	6.44
TC208	2612.20	**2612.20**	2885.65	2888.43	2858.87	2701.21	2776.85	**0.00**	10.47	10.57	9.44	3.41	6.30
TR101	1242.53	1261.10	**1242.53**	1282.80	1258.36	1287.97	1247.10	1.49	**0.00**	3.24	1.27	3.66	0.37
TR102	1126.00	1129.82	**1126.00**	1174.76	1139.31	1169.70	1127.09	0.34	**0.00**	4.33	1.18	3.88	0.10
TR103	1093.83	1096.06	**1093.83**	1140.50	1105.17	1124.98	1100.72	0.20	**0.00**	4.27	1.04	2.85	0.63
TR104	1008.83	**1008.83**	1025.37	1064.00	1029.75	1064.03	1022.70	**0.00**	1.64	5.47	2.07	5.47	1.37
TR105	1126.85	1129.40	**1126.85**	1175.27	1151.80	1177.02	1133.13	0.23	**0.00**	4.30	2.21	4.45	0.56
TR106	1108.09	**1108.09**	1109.96	1157.13	1126.67	1140.82	1109.06	**0.00**	0.17	4.43	1.68	2.95	0.09
TR107	1051.55	1052.40	**1051.55**	1095.05	1070.00	1091.03	1059.32	0.08	**0.00**	4.14	1.75	3.75	0.74
TR108	972.06	**972.06**	997.06	1031.86	998.50	1021.49	989.08	**0.00**	2.57	6.15	2.72	5.09	1.75
TR109	1018.64	1021.67	1034.90	1050.85	1043.46	1070.01	**1018.64**	0.30	1.60	3.16	2.44	5.04	**0.00**
TR110	1031.55	**1031.55**	1065.03	1073.14	1073.31	1117.15	1043.77	**0.00**	3.25	4.03	4.05	8.30	1.18
TR111	1035.51	**1035.51**	1047.43	1066.98	1053.23	1097.71	1037.00	**0.00**	1.15	3.04	1.71	6.01	0.14
TR112	962.40	**962.40**	1001.18	987.64	1007.92	1037.27	975.32	**0.00**	4.03	2.62	4.73	7.78	1.34
TR201	1467.80	**1467.80**	1544.00	1542.74	1544.67	1614.86	1524.64	**0.00**	5.19	5.11	5.24	10.02	3.87
TR202	1515.16	**1515.16**	1603.30	1601.02	1591.90	1640.28	1581.19	**0.00**	5.82	5.67	5.06	8.26	4.36
TR203	1426.66	**1426.66**	1501.54	1497.50	1479.39	1502.11	1441.09	**0.00**	5.25	4.97	3.70	5.29	1.01
TR204	1377.51	**1377.51**	1479.37	1460.39	1420.49	1447.21	1436.02	**0.00**	7.39	6.02	3.12	5.06	4.25
TR205	1460.35	**1460.35**	1544.32	1554.26	1541.49	1554.47	1507.04	**0.00**	5.75	6.43	5.56	6.45	3.20
TR206	1425.37	**1425.37**	1531.49	1520.63	1501.49	1542.17	1455.20	**0.00**	7.44	6.68	5.34	8.19	2.09
TR207	1337.63	**1337.63**	1435.11	1428.36	1406.43	1420.97	1407.38	**0.00**	7.29	6.78	5.14	6.23	5.21
TR208	1285.34	**1285.34**	1380.08	1376.17	1325.95	1336.69	1342.60	**0.00**	7.37	7.07	3.16	3.99	4.45

Table 9. *Cont.*

Instance	Best Values	Algorithms						RPI					
		QPIG	IIG	CDABC	MPSO	Q_DPIG	QIG	QPIG	IIG	CDABC	MPSO	Q_DPIG	QIG
TR209	1306.99	**1306.99**	1402.40	1393.78	1359.05	1370.90	1337.90	**0.00**	7.30	6.64	3.98	4.89	2.36
TR210	1449.77	**1449.77**	1544.08	1513.73	1493.69	1525.01	1477.03	**0.00**	6.51	4.41	3.03	5.19	1.88
TR211	1196.74	**1196.74**	1310.09	1280.97	1251.68	1282.70	1221.86	**0.00**	9.47	7.04	4.59	7.18	2.10
TRC101	1169.57	**1169.57**	1171.07	1232.29	1190.19	1227.34	1179.06	**0.00**	0.13	5.36	1.76	4.94	0.81
TRC102	1153.06	**1159.09**	1159.51	1217.49	1162.49	1225.45	1172.00	**0.00**	0.04	5.04	0.29	5.73	1.11
TRC103	1141.06	**1141.06**	1172.13	1228.53	1167.45	1213.30	1181.63	**0.00**	2.72	7.66	2.31	6.33	3.56
TRC104	1040.57	**1040.57**	1103.10	1111.57	1070.52	1096.12	1092.91	**0.00**	6.01	6.82	2.88	5.34	5.03
TRC105	1163.66	**1163.66**	1172.51	1250.77	1167.58	1202.88	1181.60	**0.00**	0.76	7.49	0.34	3.37	1.54
TRC106	1059.40	**1059.40**	1085.22	1150.81	1091.66	1138.33	1086.86	**0.00**	2.44	8.63	3.05	7.45	2.59
TRC107	1067.67	**1067.67**	1121.66	1160.09	1126.24	1175.64	1110.94	**0.00**	5.06	8.66	5.49	10.11	4.05
TRC108	1012.76	**1012.76**	1090.56	1109.06	1070.60	1091.82	1073.83	**0.00**	7.68	9.51	5.71	7.81	6.03
TRC201	1559.46	**1559.46**	1762.85	1695.61	1736.90	1723.93	1693.59	**0.00**	13.04	8.73	11.38	10.55	8.60
TRC202	1554.15	**1554.15**	1772.80	1663.15	1718.88	1703.24	1683.22	**0.00**	14.07	7.01	10.60	9.59	8.30
TRC203	1497.82	**1497.82**	1704.26	1630.99	1652.12	1620.51	1634.44	**0.00**	13.78	8.89	10.30	8.19	9.12
TRC204	1482.91	**1482.91**	1692.69	1645.44	1615.15	1629.58	1625.89	**0.00**	14.15	10.96	8.92	9.89	9.64
TRC205	1587.36	**1587.36**	1796.36	1712.06	1763.47	1744.58	1738.07	**0.00**	13.17	7.86	11.09	9.90	9.49
TRC206	1445.01	**1445.01**	1655.54	1627.73	1639.92	1620.91	1577.30	**0.00**	14.57	12.65	13.49	12.17	9.16
TRC207	1343.05	**1343.05**	1566.92	1534.93	1514.68	1495.29	1476.90	**0.00**	16.67	14.29	12.78	11.34	9.97
TRC208	1181.76	**1181.76**	1393.17	1387.37	1306.01	1361.51	1291.55	**0.00**	17.89	17.40	10.51	15.21	9.29
Mean	1514.61	**1515.62**	1638.64	1641.24	1620.19	1626.25	1589.61	**0.05**	7.49	8.02	6.31	7.50	4.49

The bold font indicates the minimum values.

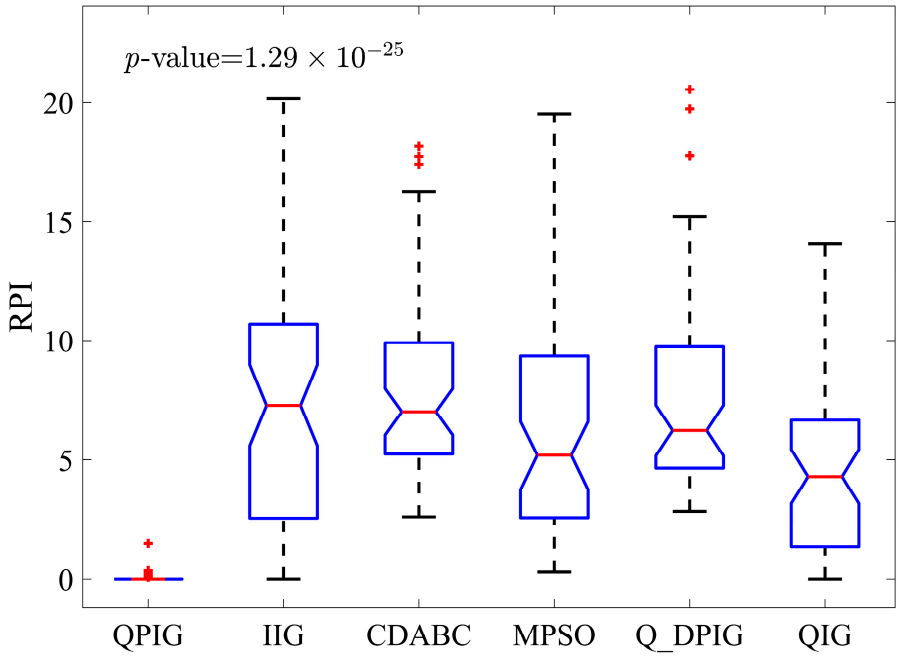


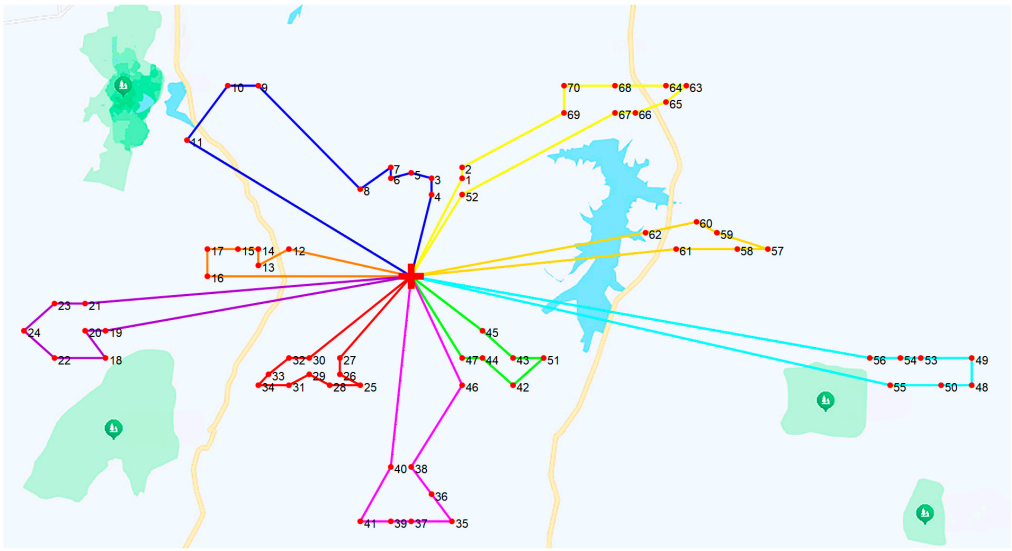
Figure 9. ANOVA comparisons of QPIG, IIG, CDABC, MPSO, Q_DPIG, and QIG.

The 56 instances are divided into four categories based on coordinates: TC101 to TC109, TC201 to TC208, TR101 to TR112, and TRC101 to TRC208. Randomly select an example from each category, and the rescue route is shown in Table 10. The first column indicates the selected instance name, the second column shows the number of robots (*Nr*), and the last column represents the rescue route. Table 10 illustrates the performance of the QPIG algorithm across various instances. Furthermore, Figure 10 presents the rescue path diagrams of these instances.

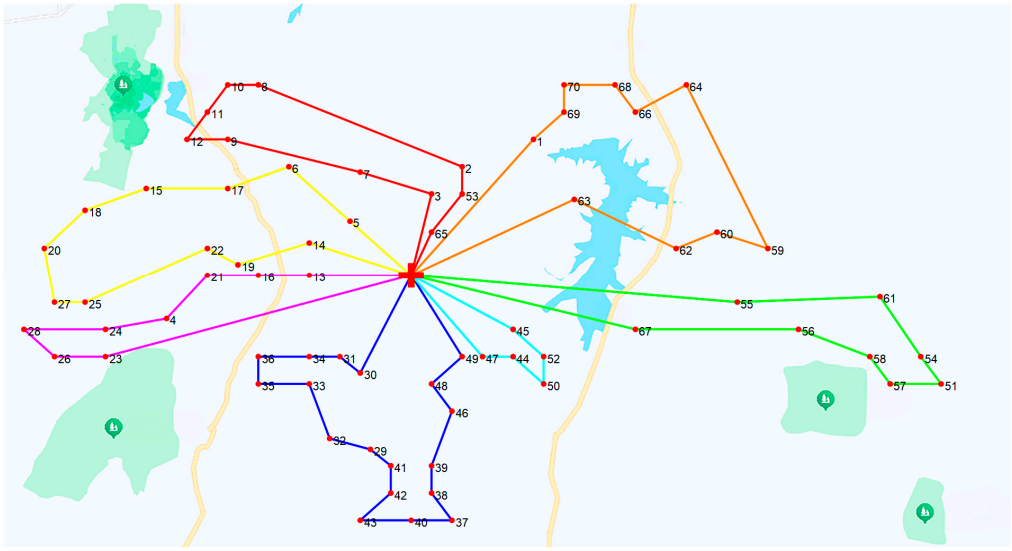
Table 10. Rescue path for the four selected instances.

Instances	*Nr*	Rescue Path
TC105	9	0→27→26→25→28→29→31→34→33→32→30→0 0→45→43→51→42→44→47→0 0→4→3→5→6→7→8→9→10→11→0 0→52→67→66→65→63→64→68→70→69→2→1→0 0→46→38→36→35→37→39→41→40→0 0→56→54→53→49→48→50→55→0 0→16→17→15→14→13→12→0 0→19→20→18→22→24→23→21→0 0→62→60→59→57→58→61→0
TC206	7	0→65→53→2→8→10→11→12→9→7→3→0 0→55→61→54→51→57→58→56→67→0 0→49→48→46→39→38→37→40→43→42→41→29→32→33→35→36→34→31→30→0 0→14→19→22→25→27→20→18→15→17→6→5→0 0→13→16→21→4→24→28→26→23→0 0→47→44→50→52→45→0 0→1→69→70→68→66→64→59→60→62→63→0
TR102	7	0→65→14→5→36→13→37→28→38→15→39→62→4→0 0→64→47→8→49→48→66→26→7→25→21→0 0→53→54→24→16→51→50→55→6→61→27→41→40→1→0 0→3→69→67→68→70→35→29→11→34→33→63→10→0 0→46→2→45→12→32→58→56→57→17→31→42→0 0→20→44→18→52→30→43→0 0→22→59→23→19→60→9→0
TRC207	8	0→50→1→4→3→5→2→70→0 0→36→61→40→56→51→55→34→68→49→0 0→9→11→31→12→10→6→8→7→0 0→63→45→69→33→39→52→54→38→59→0 0→46→17→18→16→13→14→15→44→0 0→48→41→29→30→28→26→25→27→35→65→67→58→0 0→37→53→62→43→60→66→64→0 0→42→32→23→21→19→20→22→24→47→57→0

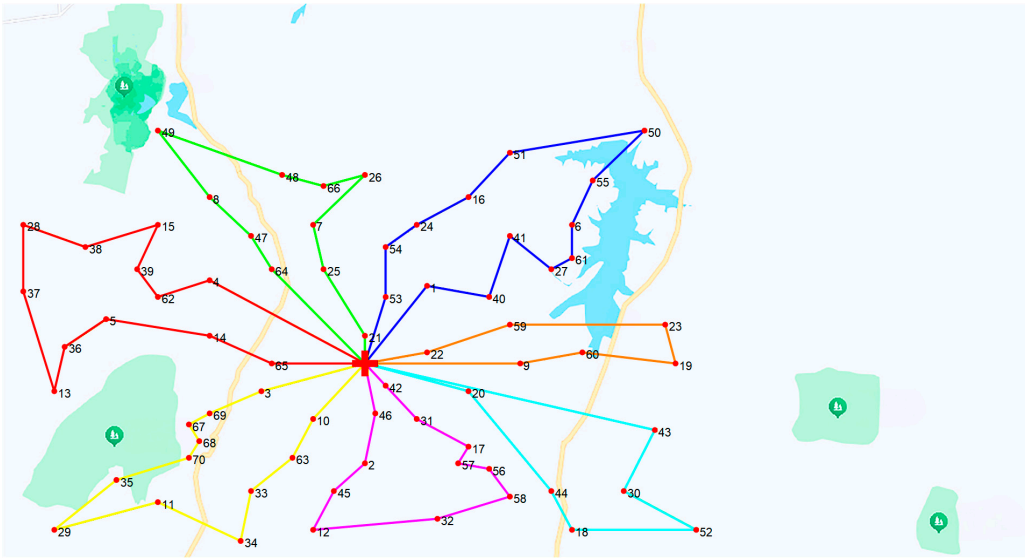
To clearly demonstrate the evolution process of all the algorithms, Figure 11 shows the convergence performance of the rescue cost on six selected instances (i.e., TC106, TC201, TR112, TR209, TRC106, and TRC208). All compared algorithms are set with a maximum runtime limit of 100 s. As shown in Figure 11, QPIG achieved the optimal initial value, indicating that the two problem-specific heuristic schemes effectively generate high-quality initial solutions. Additionally, among all the compared algorithms, the QPIG algorithm has a better convergence speed and the lowest total rescue cost, which reflects the effectiveness of the competition-oriented destruction-reconstruction mechanism, Q-learning-based local search strategy, and historical information-based constructive strategy. In summary, the simulation results indicate that QPIG can provide higher-quality solutions when solving the MRRPP problem.



(**a**) Visualization of instance TC105.

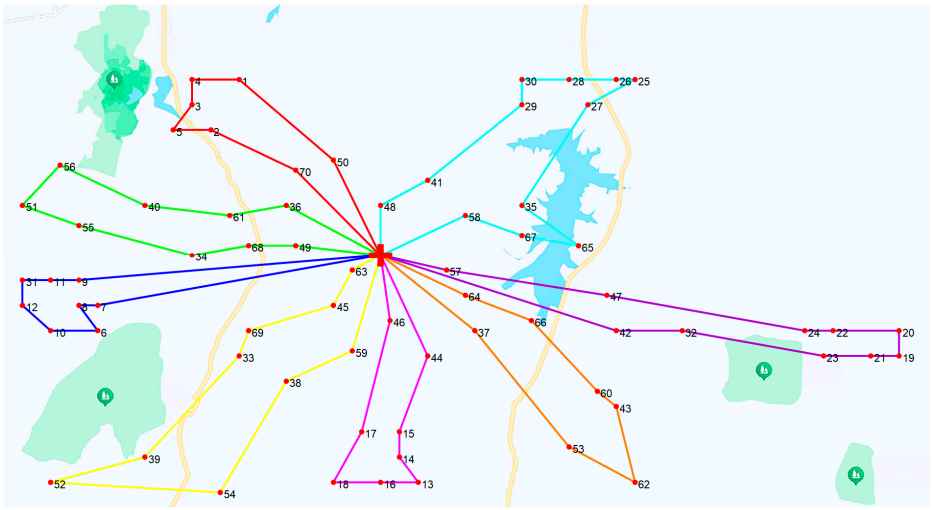


(**b**) Visualization of instance TC206.



(**c**) Visualization of instance TR102.

Figure 10. *Cont*.



(**d**) Visualization of instance TRC207.

Figure 10. Rescue paths corresponding to four instances.

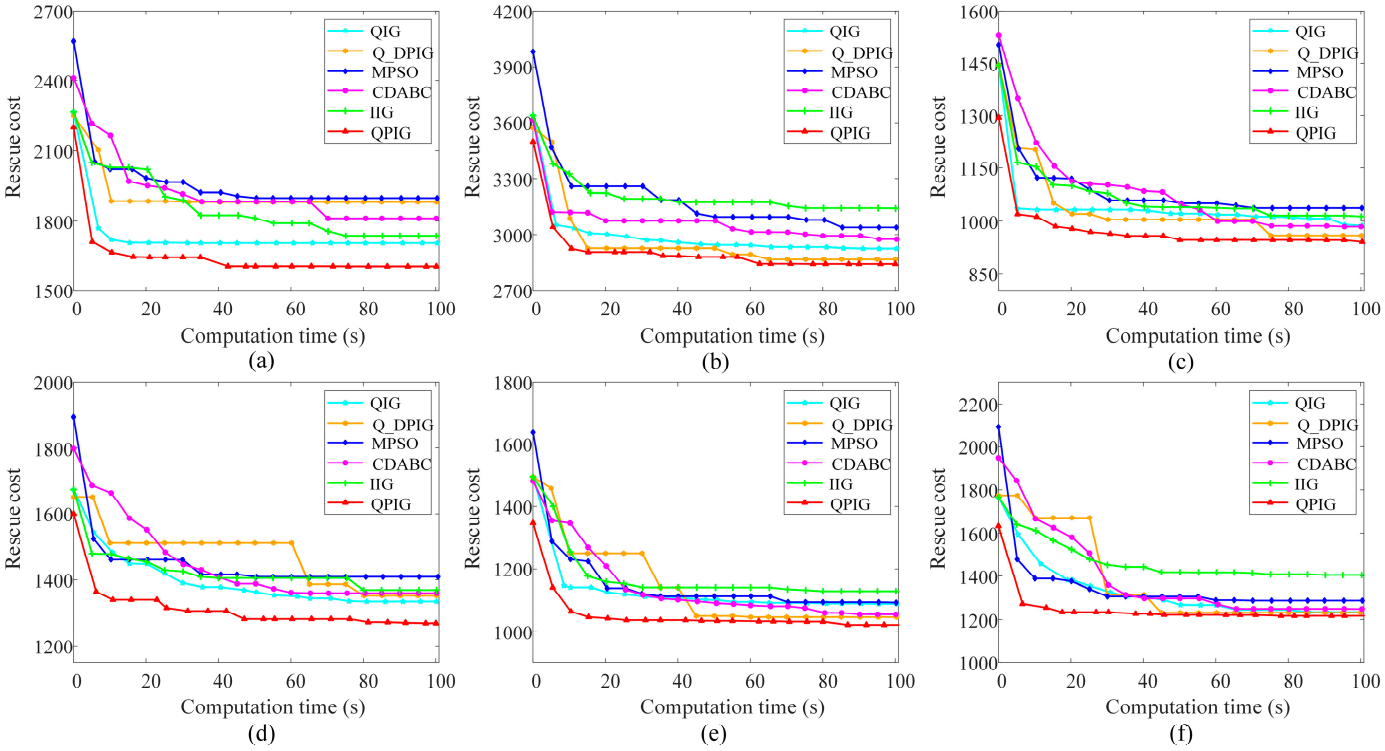


Figure 11. Convergence performance for all algorithms across six instances. (**a**) Instance TC106; (**b**) instance TC201; (**c**) instance TR101; (**d**) instance TR112; (**e**) instance TRC106; (**f**) instance TRC208.

5.6. Statistical Analysis

The Friedman test was employed to analyze the performance differences among IIG, CDABC, MPSO, Q_DPIG, QIG, and QPIG. The null hypothesis assumes no significant performance differences between the algorithms, with a significance level set at $\alpha = 0.05$. The null hypothesis is rejected when the p-value is less than α. Figure 12 visually presents the significant differences between the algorithms under the two evaluation metrics, RPI and rescue cost. Table 11 provides more detailed data, including rank values, the number of instances (N), mean, standard deviation, minimum, and maximum.

In Table 11 for the RPI metric, the results show that QPIG, IIG, CDABC, MPSO, Q_DPIG, and QIG achieve ranks of 1.16, 4.36, 4.77, 3.64, 4.52, and 2.55, respectively. QPIG

demonstrates the best performance with a minimum value of 0, a maximum of 1.49, a standard deviation of 0.21, and a mean of 0.05. Similarly, for the rescue cost metric, the rankings remain consistent, with QPIG achieving the top results, with a minimum of 962.40, a maximum of 2823.61, a standard deviation of 543.84, and a mean of 1515.62. These results highlight QPIG's outstanding performance in both the RPI and rescue cost metrics, supporting its reliability and superiority in addressing the MRPP problem.

To further validate the performance distinctions among the five algorithms, we applied the Wilcoxon signed-rank tests to both the RPI and rescue cost metrics. The detailed results, presented in Table 12, indicate statistically significant differences at the $\alpha = 0.05$ level across all 56 instances, clearly demonstrating the variability in the performance of these algorithms.

Table 11. Friedman test results ($\alpha = 0.05$) for the RPI and the rescue cost.

Algorithms	Ranks	N	RPI				The Rescue Cost			
			Mean	Std. Deviation	Min	Max	Mean	Std. Deviation	Min	Max
QPIG	**1.16**	56	**0.05**	**0.21**	**0.00**	**1.49**	**1515.62**	**543.84**	**962.40**	**2823.61**
IIG	4.36	56	7.49	5.96	**0.00**	24.96	1638.64	617.56	997.06	3059.34
CDABC	4.77	56	8.02	3.86	2.62	18.18	1641.24	601.19	987.64	3030.75
MPSO	3.64	56	6.31	4.59	0.29	19.52	1620.19	610.65	998.50	3025.89
Q_DPIG	4.52	56	7.50	4.10	2.85	20.55	1626.25	566.63	1021.49	2959.29
QIG	2.55	56	4.49	3.39	**0.00**	14.06	1589.61	590.18	975.32	2970.72

The bold font indicates the minimum values.

Table 12. Results of the Wilcoxon signed-rank test for the RPI and the rescue cost.

QPIG vs.	RPI			The Rescue Cost		
	p-Value	Significant	Decision	*p*-Value	Significant	Decision
IIG	0.00	Yes	Reject H_0	0.00	Yes	Reject H_0
CDABC	0.00	Yes	Reject H_0	0.00	Yes	Reject H_0
MPSO	0.00	Yes	Reject H_0	0.00	Yes	Reject H_0
Q_DPIG	0.00	Yes	Reject H_0	0.00	Yes	Reject H_0
QIG	0.00	Yes	Reject H_0	0.00	Yes	Reject H_0

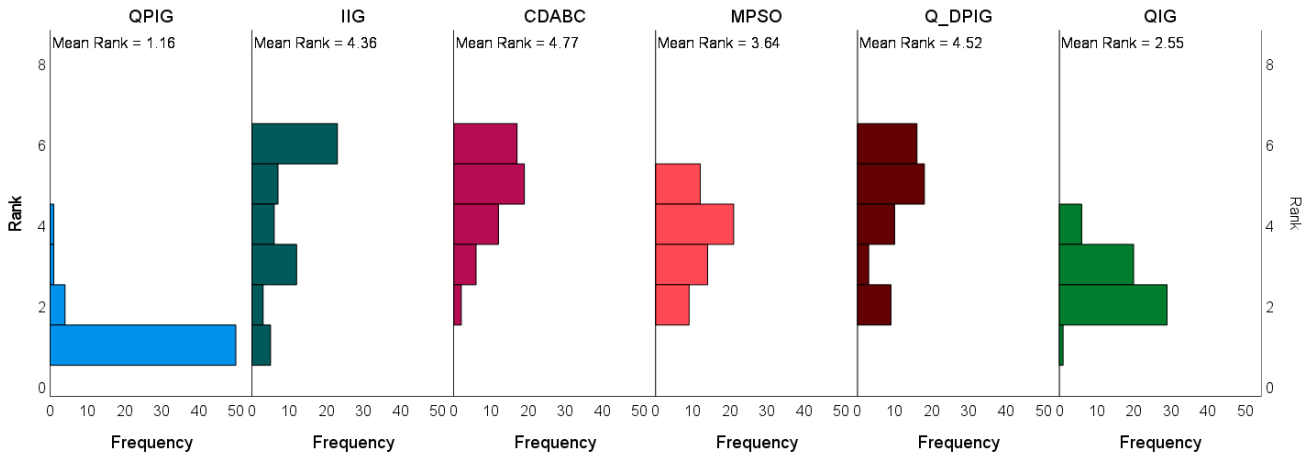


Figure 12. Related-samples Friedman's two-way analysis of variance by ranks.

6. Conclusions and Future Work

This study presents the QPIG algorithm to solve the MRRPP problem. To obtain a high-quality initial population, two problem-specific heuristic schemes are designed. A competition-oriented destruction-reconstruction mechanism is applied, which includes the

history-based exploration strategy, the collective intelligence-driven strategy, and the elite-oriented optimization strategy to enhance global exploration ability. A Q-learning-based local search strategy is implemented to select six operators to improve the algorithm's exploitation ability. Additionally, a historical information-based constructive strategy is investigated to accelerate the algorithm's convergence. Finally, experiments on 56 instances are conducted using the QPIG algorithm and other compared algorithms. This confirms the effectiveness of the QPIG algorithm in tackling the MRRPP problem. In addition, we analyzed the convergence curve of the algorithm and found that the proposed QPIG can achieve results close to the optimal solution within 10 s and 20 s, and the difference from the optimal value is usually not more than 5%. This result indicates that our method not only converges rapidly but also has good stability and efficiency, making it suitable for real-time applications with high time requirements. Therefore, even with the continuous improvement of scale or map resolution, QPIG can maintain stable computing behavior, converge within a reasonable time range, and provide near-optimal solutions, indicating its good scalability.

The limitations of this work include the following: (1) The QPIG algorithm does not adequately account for dynamic obstacles in rescue scenarios. (2) The proposed QPIG lacks consideration of the priority of task points, which may lead to neglecting the priority processing of emergency task points. (3) This study does not consider robot malfunctions or damage, which could disrupt the vital search process.

Future research will investigate the following directions: (1) The influence of environmental conditions, particularly the challenges introduced by moving obstacles, will be taken into account. (2) As mentioned in [43], we will address the rescheduling problem when robots encounter damage, ensuring that tasks are reassigned and operations continue efficiently. (3) We will focus on incorporating priority approaches to handle urgent rescue tasks in path planning.

Author Contributions: Conceptualization, M.L. and P.D.; methodology, M.L.; software, M.L.; validation, M.L.; investigation, M.L. and P.D.; writing—original draft preparation, M.L.; writing—review and editing, P.D.; supervision, P.D.; funding acquisition, P.D. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Open Research Fund of Key Laboratory of Machine Intelligence and System Control, Ministry of Education (MISC-202411), the Natural Science Foundation of Shandong Province, China (ZR2021MD090), and the Discipline with Strong Characteristics of Liaocheng University-Intelligent Science and Technology, China (319462208).

Data Availability Statement: The original contributions presented in the study are included in the article; further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Nomenclature

0	Rescue center
N	Set of the rescue center and rescue points
T	Set of rescue points
R	Set of robots
i, j	Indices of rescue points
k	Index of robots
d_{ij}	Distance between rescue points T_i and T_j
w_l	Weight coefficient for path length
w_n	Weight coefficient for the number of robots

w_t	Weight coefficient for rescue time
w_r	Weight coefficient for task utility
a_i	Start time of the time window for the rescue point T_i
b_i	End time of the time window for the rescue point T_i
dr_i	Material demand at the rescue point T_i
C_k^{max}	Maximum carrying capacity of robot R_k
U_i	Initial utility of the task at the rescue point T_i
μ_i	Time decay factor affecting the task at the rescue point T_i
t_{ik}	Service start time of robot R_k at rescue point T_i
τ_n^e	Completion time of the last rescue point task
τ_1^s	Start time of the first rescue point task
E_k	Battery consumption of robot R_k while executing the task at rescue point T_i
$d_{i,k}$	Distance traveled by robot R_k while executing the task at rescue point T_i
E_a	Battery consumption per unit distance traveled by the robot
ξ	Reserve battery capacity
Q_k	Total battery capacity of robot R_k

References

1. Pelton, J.N. Natural disasters. In *Space Systems and Sustainability: From Asteroids and Solar Storms to Pandemics and Climate Change*; Springer Nature: Berlin/Heidelberg, Germany, 2021; pp. 193–207. [CrossRef]
2. Ransikarbum, K.; Mason, S.J. A bi-objective optimisation of post-disaster relief distribution and short-term network restoration using hybrid NSGA-II algorithm. *Int. J. Prod. Res.* **2022**, *60*, 5769–5793. [CrossRef]
3. Surmann, H.; Daun, K.; Schnaubelt, M.; von Stryk, O.; Patchou, M.; Böcker, S.; Wietfeld, C.; Quenzel, J.; Schleich, D.; Behnke, S. Lessons from robot—Assisted disaster response deployments by the German rescue robotics center task force. *J. Field Robot.* **2024**, *41*, 782–797. [CrossRef]
4. Martinez-Alpiste, I.; Golcarenarenji, G.; Wang, Q.; Alcaraz-Calero, J.M. Search and rescue operation using UAVs: A case study. *Expert Syst. Appl.* **2021**, *178*, 114937. [CrossRef]
5. Al-Hussaini, S.; Gregory, J.M.; Gupta, S.K. Generating task reallocation suggestions to handle contingencies in human-supervised multi-robot missions. *IEEE Trans. Autom. Sci. Eng.* **2023**, *21*, 367–381. [CrossRef]
6. Akay, R.; Yildirim, M.Y. Multi-strategy and self-adaptive differential sine–cosine algorithm for multi-robot path planning. *Expert Syst. Appl.* **2023**, *232*, 120849. [CrossRef]
7. Lyu, M.; Zhao, Y.; Huang, C.; Huang, H. Unmanned aerial vehicles for search and rescue: A survey. *Remote Sens.* **2023**, *15*, 3266. [CrossRef]
8. Cui, X.; Yang, K.; Wang, X.; Duan, P. An improved iterated greedy algorithm for solving collaborative helicopter rescue routing problem with time window and limited survival time. *Algorithms* **2024**, *17*, 431. [CrossRef]
9. Lin, S.; Geng, N.; Sun, J.; Zhang, Y. A grouping method based on improved PSO for task allocation in rescue environment. In Proceedings of the 2019 IEEE Congress on Evolutionary Computation (CEC), Wellington, New Zealand, 10–13 June 2019; pp. 619–626. [CrossRef]
10. Geng, N.; Chen, Z.; Nguyen, Q.A.; Gong, D. Particle swarm optimization algorithm for the optimization of rescue task allocation with uncertain time constraints. *Complex Intell. Syst.* **2021**, *7*, 873–890. [CrossRef]
11. Huang, J.; Song, Q.; Xu, Z. Multi robot cooperative rescue based on two-stage task allocation algorithm. *J. Phys. Conf. Ser.* **2022**, *2310*, 012091. [CrossRef]
12. Wang, X.; Duan, P.; Meng, L.; Yang, K. An improved iterated greedy algorithm for solving rescue robot path planning problem with limited survival time. *Comput. Mater. Contin.* **2024**, *80*, 931–947. [CrossRef]
13. Liu, W.; Zhou, Y.; Liu, W.; Qiu, J.; Xie, N.; Chang, X.; Chen, J. A hybrid ACS-VTM algorithm for the vehicle routing problem with simultaneous delivery & pickup and real-time traffic condition. *Comput. Ind. Eng.* **2021**, *162*, 107747. [CrossRef]
14. Chen, Y.; Huang, K. Solving Heterogeneous Capacitated Vehicle Routing Problem Assisted with Deep Reinforcement Learning. In Proceedings of the 2025 International Conference on Advanced Machine Learning and Data Science (AMLDS), Tokyo, Japan, 19–21 July 2025; pp. 472–477. [CrossRef]
15. Wang, H.; Chen, W. Multi-robot path planning with due times. *IEEE Robot. Autom. Lett.* **2022**, *7*, 4829–4836. [CrossRef]
16. Asfora, B.A.; Banfi, J.; Campbell, M. Mixed-integer linear programming models for multi-robot non-adversarial search. *IEEE Robot. Autom. Lett.* **2020**, *5*, 6805–6812. [CrossRef]
17. Velasco, L.; Guerrero, H.; Hospitaler, A. A literature review and critical analysis of metaheuristics recently developed. *Arch. Comput. Methods Eng.* **2024**, *31*, 125–146. [CrossRef]

18. Wang, Y.; Wang, Y.; Han, Y.; Gao, K.; Li, J.; Wang, Y. Enhancing distributed blocking flowshop group scheduling: Theoretical insight and application of an iterated greedy algorithm with idle time insertion and rapid evaluation mechanisms. *Expert Syst. Appl.* **2025**, *271*, 126600. [CrossRef]
19. Zou, W.; Pan, Q.; Tasgetiren, M.F. An effective iterated greedy algorithm for solving a multi-compartment AGV scheduling problem in a matrix manufacturing workshop. *Appl. Soft Comput.* **2021**, *99*, 106945. [CrossRef]
20. Liu, J.; Pan, Q.; Li, W.; Wang, B. A novel elite-preserving iterated greedy algorithm with Q-learning for cascaded flowshop joint scheduling problem. *Expert Syst. Appl.* **2026**, *297*, 129512. [CrossRef]
21. Wang, Y.; Han, Y.; Wang, Y.; Li, J.; Gao, K.; Liu, Y. An effective two-stage iterated greedy algorithm for distributed flowshop group scheduling problem with setup time. *Expert Syst. Appl.* **2023**, *233*, 120909. [CrossRef]
22. Xu, X.; Zhang, L.; Trovati, M.; Palmieri, F.; Asimakopoulou, E.; Johnny, O.; Bessis, N. PERMS: An efficient rescue route planning system in disasters. *Appl. Soft Comput.* **2021**, *111*, 107667. [CrossRef]
23. Morin, M.; Abi-Zeid, I.; Quimper, C.-G. Ant colony optimization for path planning in search and rescue operations. *Eur. J. Oper. Res.* **2023**, *305*, 53–63. [CrossRef]
24. Wen, H.; Shi, Y.; Wang, S.; Chen, T.; Di, P.; Yang, L. Route planning for UAVs maritime search and rescue considering the targets moving situation. *Ocean Eng.* **2024**, *310*, 118623. [CrossRef]
25. Alrayes, F.S.; Dhahbi, S.; Alzahrani, J.S.; Mehanna, A.S.; Al Duhayyim, M.; Motwakel, A.; Yaseen, I.; Atta Abdelmageed, A. Enhanced search-and-rescue optimization-enabled secure route planning scheme for internet of drones environment. *Appl. Sci.* **2022**, *12*, 7950. [CrossRef]
26. Liu, K.; Liu, D.; An, K.; Liudandan, L. Efficient multi-objective path planning for complex disaster environments: Hierarchical target filtering and ranking optimization strategies. *Intell. Serv. Rob.* **2025**, *18*, 13–25. [CrossRef]
27. Bouamama, S.; Blum, C.; Boukerram, A. A population-based iterated greedy algorithm for the minimum weight vertex cover problem. *Appl. Soft Comput.* **2012**, *12*, 1632–1639. [CrossRef]
28. Miao, Z.; Guo, H.; Pan, Q.; Peng, C.; Xu, Z. A reinforcement learning-enhanced multi-objective iterated greedy algorithm for weeding-robot operation scheduling problems. *Expert Syst. Appl.* **2025**, *263*, 125760. [CrossRef]
29. Zou, W.; Zou, J.; Sang, H.; Meng, L.; Pan, Q. An effective population-based iterated greedy algorithm for solving the multi-AGV scheduling problem with unloading safety detection. *Inf. Sci.* **2024**, *657*, 119949. [CrossRef]
30. Wang, C.; Pan, Q.; Sang, H. The cascaded flowshop joint scheduling problem: A mathematical model and population-based iterated greedy algorithm to minimize total tardiness. *Robot. Comput. Integr. Manuf.* **2024**, *88*, 102747. [CrossRef]
31. Zhao, H.; Pan, Q.; Gao, K. A cooperative population-based iterated greedy algorithm for distributed permutation flowshop group scheduling problem. *Eng. Appl. Artif. Intell.* **2023**, *125*, 106750. [CrossRef]
32. Wang, Y.; Wang, Y.; Han, Y.; Li, J.; Gao, K. A rapid population-based iterated greedy for distributed blocking group flowshop scheduling with delivery time windows under multiple processing time scenarios. *Comput. Ind. Eng.* **2025**, *202*, 110949. [CrossRef]
33. Li, J.; Li, R.; Li, J.; Yu, X.; Xu, Y. A multi-dimensional co-evolutionary algorithm for multi-objective resource-constrained flexible flowshop with robotic transportation. *Appl. Soft Comput.* **2025**, *170*, 112689. [CrossRef]
34. Zhao, F.; Zhuang, C.; Wang, L.; Dong, C. An iterative greedy algorithm with Q-learning mechanism for the multiobjective distributed no-idle permutation flowshop scheduling. *IEEE Trans. Syst. Man Cybern. Syst.* **2024**, *54*, 3207–3219. [CrossRef]
35. Ren, Y.; Gao, K.; Fu, Y.; Li, D.; Suganthan, P.N. Ensemble artificial bee colony algorithm with Q-learning for scheduling bi-objective disassembly line. *Appl. Soft Comput.* **2024**, *155*, 111415. [CrossRef]
36. Luo, C.; Gong, W.; Ming, F.; Lu, C. A Q-learning memetic algorithm for energy-efficient heterogeneous distributed assembly permutation flowshop scheduling considering priorities. *Swarm Evol. Comput.* **2024**, *85*, 101497. [CrossRef]
37. Chen, T.; Miao, Z.; Li, W.; Pan, Q. A learning-based memetic algorithm for a cooperative task allocation problem of multiple unmanned aerial vehicles in smart agriculture. *Swarm Evol. Comput.* **2024**, *91*, 101694. [CrossRef]
38. Fang, W.; Liao, Z.; Bai, Y. Improved ACO algorithm fused with improved Q-Learning algorithm for Bessel curve global path planning of search and rescue robots. *Robot. Auton. Syst.* **2024**, *182*, 104822. [CrossRef]
39. Zhan, H.; Zhang, Y.; Huang, J.; Song, Y.; Xing, L.; Wu, J.; Gao, Z. A reinforcement learning-based evolutionary algorithm for the unmanned aerial vehicles maritime search and rescue path planning problem considering multiple rescue centers. *Memet. Comput.* **2024**, *16*, 373–386. [CrossRef]
40. Li, J.; Du, Y.; Gao, K.; Duan, P.; Gong, D.; Pan, Q.; Suganthan, P.N. A hybrid iterated greedy algorithm for a crane transportation flexible job shop problem. *IEEE Trans. Autom. Sci. Eng.* **2021**, *19*, 2153–2170. [CrossRef]
41. Li, C.; Zhu, Y.; Lee, K.Y. Route optimization of electric vehicles based on reinsertion genetic algorithm. *IEEE Trans. Transp. Electrif* **2023**, *9*, 3753–3768. [CrossRef]
42. Zhang, J.; Yu, M.; Feng, Q.; Leng, L.; Zhao, Y. Data-Driven robust optimization for solving the heterogeneous vehicle routing problem with customer demand uncertainty. *Complexity* **2021**, *2021*, 6634132. [CrossRef]
43. Wang, X.; Zou, W.; Meng, L.; Zhang, B.; Li, J.; Sang, H. Effective metaheuristic and rescheduling strategies for the multi-AGV scheduling problem with sudden failure. *Expert Syst. Appl.* **2024**, *250*, 123473. [CrossRef]

44. Duan, P.; Yu, Z.; Gao, K.; Meng, L.; Han, Y.; Ye, F. Solving the multi-objective path planning problem for mobile robot using an improved NSGA-II algorithm. *Swarm Evol. Comput.* **2024**, *87*, 101576. [CrossRef]
45. Guo, H.; Miao, Z.; Ji, J.; Pan, Q. An effective collaboration evolutionary algorithm for multi-robot task allocation and scheduling in a smart farm. *Knowl. Based Syst.* **2024**, *289*, 111474. [CrossRef]
46. Poy, Y.L.; Darmaraju, S.; Kwan, B.-H. Multi-robot path planning using modified particle swarm optimization. In Proceedings of the 2023 IEEE International Conference on Automatic Control and Intelligent Systems (I2CACIS), Shah Alam, Malaysia, 17 June 2023; pp. 225–230. [CrossRef]
47. Qin, H.; Han, Y.; Chen, Q.; Wang, L.; Wang, Y.; Li, J.; Liu, Y. Energy-efficient iterative greedy algorithm for the distributed hybrid flow shop scheduling with blocking constraints. *IEEE Trans. Emerg. Top. Comput. Intell.* **2023**, *7*, 1442–1457. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.